FINAL REPORT

Team Members:

Sezer Ataş

Muhammed Yusuf Özcan

Alperen Berke Çetinkaya

Contents

1. METHODOLOGY

1.1. Study Scope and Methodological Objective

This study presents an offline, case-specific brain tumor MRI decision-support prototype that transforms segmentation outputs into quantitative, anatomically grounded, and clinically readable artifacts. The system is not designed as an autonomous diagnostic tool. Instead, it acts as a post-segmentation interpretation framework that connects tumor masks to measurable descriptors, approximate neuroanatomic explanations, structured question-answering outputs, and draft clinical reports.

The central methodological principle of the project is grounded generation. In practice, this means that the system should remain anchored to explicit case data while still producing outputs that are readable and useful for clinicians. For that reason, the architecture deliberately separates the deterministic measurement layer from the controlled narrative layer. Quantitative measurements, lesion counts, and structured descriptors are produced deterministically, whereas natural-language fluency is added selectively through a constrained local language model.

1.2. Overall System Workflow

At a high level, the pipeline can be summarized as follows:

MRI case package → segmentation mask → quantitative tumor metrics → atlas-aware anatomic descriptors → grounded report and QA generation

The workflow begins with the organization of MRI sequences at the case level. After segmentation is available, the mask is converted into clinically relevant volumetric and lesion-centric measurements. These measurements are then enriched with approximate anatomic localization through atlas-based mapping. Finally, the resulting structured case representation is used for report generation and question answering.

Methodologically, this is a hybrid system. The numeric truth layer is deterministic, reproducible, and traceable to structured fields. The explanatory layer is partially generative but constrained by curated descriptors, safety rules, and validation logic.

1.3. Case Discovery and Data Preparation

Case-Level Packaging

Each case is stored in a dedicated folder containing multimodal MRI sequences, typically including T1, T1ce, T2, and FLAIR images in NIfTI format. The system scans the case directory, identifies available files, and derives a minimal study-level description of the input package. This stage does not perform medical reasoning; its purpose is to create a stable and reproducible case context for all downstream stages.

Structured Patient Context

The discovered imaging inputs are transformed into a structured patient_context representation. This object stores identifiers such as patient ID, study ID, study date, modality, and imaging availability summaries. It also acts as the shared case-level substrate for both reporting and question answering. Because both downstream components read from the same context object, the system remains case-locked: outputs are produced from the currently loaded case package rather than from unconstrained general medical knowledge.

1.4. Segmentation Layer

Segmentation as a Prerequisite Layer

The segmentation stage follows an nnU-Net-style inference workflow. MRI sequences are prepared in the expected channel format, and segmentation masks are generated as NIfTI outputs. In this project, segmentation is treated as a prerequisite computational layer rather than the main scientific contribution. This design choice is consistent with the literature, where nnU-Net has become a strong baseline for biomedical segmentation and where multimodal glioma MRI segmentation has been extensively studied through the BraTS benchmark ecosystem.

Methodological Role of Segmentation

The main contribution of this study is therefore not a new segmentation model. Instead, the contribution lies in what happens after segmentation: the conversion of a mask into measurable, explainable, and reportable clinical artifacts. This positioning is important, because it defines the project as an integrative interpretation system rather than as a benchmark-optimizing segmentation paper.

1.5. Quantitative Tumor Metrics Extraction

From Mask to Physical Measurements

Once a segmentation mask is available, the system computes quantitative tumor metrics directly from it. This stage is one of the most important parts of the methodology because it converts a voxelwise label image into clinically interpretable measurements. The mask is read together with voxel spacing information so that all volumetric values can be expressed in physical units such as cubic millimeters and cubic centimeters.

Core Metrics

The extracted metrics include:

- total segmented tumor volume
- maximum segmented lesion diameter
- label-specific component volumes
- lesion list
- merged lesion-group count
- segmented component count

Label-specific burden is computed separately for enhancing tumor, non-enhancing tumor, and edema. These quantities are preserved both as absolute measurements and as proportions of the total segmented burden.

Lesion Groups Versus Raw Components

An important design decision is the distinction between lesion groups and segmented components. The system does not simply count every connected fragment as an independent lesion. Instead, compatible tumor labels are merged first so that the resulting lesion-group count better reflects biologically meaningful lesion structure. This reduces the risk of overcalling multifocal disease when a single heterogeneous lesion appears fragmented in the segmentation mask. In addition, very small fragments can be filtered using a minimum voxel threshold so that obvious segmentation debris does not dominate the case interpretation.

1.6. Lesion-Centric Representation

Beyond global case-level summaries, the system preserves a lesion-centric data model. Each meaningful component in the segmentation is represented in a lesion_list with fields such as lesion ID, label type, volume, maximum diameter, centroid, and anatomic descriptor. This design is methodologically important because it avoids collapsing the entire case into a single summary number.

The lesion-centric representation supports clinically meaningful downstream questions, including:

- identification of the largest lesion
- characterization of the dominant component
- assessment of whether the burden is concentrated in one dominant lesion with satellite groups
- localization of specific lesion components

This intermediate representation is also reused in both the reporting and QA stages, improving consistency across system outputs.

1.7. Atlas-Backed Anatomic Mapping

Approximate Neuroanatomic Localization

Quantitative measurements alone are not sufficient for clinically useful explanation. For this reason, the pipeline includes an atlas-backed anatomic mapping stage. Lesions are approximately aligned into standard space and sampled against the Harvard-Oxford atlas. This produces descriptors such as hemisphere, lobe, depth, superior-middle-inferior level, and, when supported, atlas-linked regional labels.

Uncertainty-Aware Localization Language

The project does not treat these localization outputs as exact anatomical truth. Instead, atlas confidence and registration quality are explicitly preserved and then propagated into the final language layer. When localization support is limited, the final wording becomes more cautious and uses terms such as "approximate atlas-supported overlap." This is methodologically significant because uncertainty is modeled as part of the pipeline itself rather than hidden beneath a single definitive region name.

1.8. Registration Quality Awareness

Atlas mapping is not treated as a binary success-or-failure step. Instead, the system retains registration quality signals, overlap-related indicators, and related mapping quality metadata. These values do not function merely as hidden debug fields; they inform how strongly localization statements may be expressed in the final outputs.

When localization support is relatively strong, the report may describe the dominant component as atlas-backed but approximate. When support is weak, the language is downgraded accordingly. This makes confidence-sensitive reporting part of the explanatory design rather than an afterthought.

1.9. Structured Descriptor Layer

Compact Intermediate Representation

The system does not pass the entire raw case context directly into the language model. Instead, it builds a compact report_descriptor containing semantically meaningful intermediate abstractions such as:

- case overview
- segmentation overview
- component profile
- dominant lesion summary
- spatial distribution
- semantic segmentation summary
- narrative hints

Why the Descriptor Layer Matters

This descriptor layer serves several purposes at once. It reduces prompt size, improves semantic organization, lowers truncation risk, and constrains the language model to grounded intermediate variables rather than loosely structured raw metadata. In practice, this is one of the main reasons the system maintains better numerical discipline than a fully free-form report generator.

This design is also aligned with recent grounded medical report-generation literature, where explainable intermediate representations are used to stabilize downstream natural-language output.

1.10. Hybrid Report Generation

Division of Responsibilities

The reporting methodology is intentionally hybrid. The deterministic layer is responsible for report structure, mandatory section headings, quantitative findings, safety-critical wording, and disclaimer boundaries. The language model is then used in a constrained role to improve fluency in selected narrative sections, especially Clinical Context, Findings, and Impression.

Report Structure

The report is organized under fixed high-level sections:

- Clinical Context
- Findings

- Impression
- Limitations
- Suggested Clinical Correlation
- Disclaimer

Within Findings, the system may further present localization, regional distribution, enhancement pattern, multiplicity, and related automatically derived descriptors. In current versions of the project, heuristic burden-related statements are deliberately softened and presented as secondary automated flags rather than direct radiologic measurements.

Soft Repair Instead of Hard Rejection

An important refinement in the report pipeline is the move from hard rejection to soft repair. Earlier versions of the system tended to discard generated narrative entirely when it contained unsafe or overly strong wording. In the current design, the report layer first attempts to repair such text while preserving useful narrative content. This produces a more readable report without giving up deterministic control over core measurements and safety boundaries.

1.11. Controlled Question Answering

QA Decision Process

The QA engine follows a structured pipeline:

- normalize the incoming question
- classify the question type
- retrieve the relevant structured fields
- generate a deterministic answer when possible
- optionally add constrained narrative support
- attach evidence IDs, confidence, and safety notes

Conservative Behavior for High-Risk Questions

For quantitative questions, answers are produced directly from structured metrics. For higher-risk questions, especially those involving diagnosis, treatment, prognosis, chemotherapy, surgery, or cure rate, the system applies explicit guardrails. In these situations, the system either refuses to make a definitive claim or explains what is known from the current case package and what additional information would be required for valid clinical decision-making.

This makes the QA layer intentionally stricter than the report layer. Methodologically, that asymmetry is deliberate: reports benefit from broader narrative continuity, whereas question answering must prioritize direct evidence linkage, traceability, and conservative safety behavior.

1.12. Evidence Tracking and Evaluation

Evidence-Linked Outputs

A major strength of the methodology lies in evidence tracking. Each QA response stores the answer text, evidence identifiers, confidence, and safety notes. Evidence identifiers can then be resolved back to the exact structured context fields from which the answer was derived. This gives the project an auditable path from output text back to case-level source data.

Run-Level Artifacts

Each report run is accompanied by a full artifact set, including:

- report draft
- QA outputs
- evidence mappings
- quality summaries
- evaluation logs
- comparison outputs
- run metadata

The run metadata additionally stores prompt policy, model version, question-set provenance, and run fingerprinting information. This transforms the system from a one-off demo into a reproducible workflow.

Evaluation Scope

The evaluation layer measures report and QA quality through metrics such as:

- section completeness
- QA evidence coverage
- unsupported statement rate
- contradiction detection
- readability
- lexical diversity

These metrics do not prove clinical validity by themselves, but they do provide a structured way to assess consistency, traceability, and output quality at the system level.

1.13. Methodological Positioning in the Literature

Within the existing literature, the project should be positioned as an integrative contribution rather than a foundational algorithmic one. It does not introduce a new segmentation architecture beyond established approaches such as nnU-Net, nor does it attempt to outperform benchmark-oriented segmentation studies such as BraTS.

Its contribution lies instead in the integration of:

- segmentation-derived quantitative metrics
- lesion-centric representation
- atlas-aware explanation
- structured reporting
- grounded question answering
- safety controls
- reproducible artifact generation

Compared with structured reporting frameworks such as BT-RADS, the system is less clinically mature but more automated at the case-processing level. Compared with LLM-first report-generation approaches, it is more conservative and more tightly grounded in deterministic case

data. Compared with segmentation-only studies, it contributes less at the model-innovation level but more at the level of explainability and downstream usability.

1.14. Limitations

Several methodological limitations remain. First, the system does not directly measure mass effect or midline shift through dedicated image analysis; these remain heuristic secondary flags rather than validated imaging biomarkers. Second, atlas localization is approximate and may become unstable when registration quality is poor. Third, the report generator still retains some template structure, especially around quantitative support lines and mandatory safety statements. Fourth, all downstream explanation quality depends on segmentation quality. If the mask is incorrect, the explanation may remain internally consistent while still being clinically misleading.

For these reasons, the system should be presented as a research or educational decision-support prototype rather than as a clinical diagnostic system.

1.15. Conclusion of the Methodological Design

In summary, the methodology of this project is built around a hybrid, safety-aware, segmentation-driven interpretation pipeline for brain tumor MRI. Its key contribution is the integration of deterministic quantitative analysis, approximate atlas-based localization, controlled narrative generation, evidence-linked QA, and reproducible evaluation into a single offline workflow.

This makes the project well suited to a final-year engineering context: not because it introduces a completely new algorithmic paradigm, but because it meaningfully combines multiple strands of current literature into a practical and explainable prototype.

1.16. Lung Nodule Segmentation Methodology

This section describes the methodology used for automated lung nodule segmentation on CT images, complementing the brain tumor segmentation pipeline described in Sections 4–5. The evaluation was performed on the Medical Segmentation Decathlon Task06_Lung dataset, consisting of 63 chest CT scans with annotated lung nodule masks.

1.17. Dataset

Source: Medical Segmentation Decathlon, Task06_Lung (Simpson et al., 2019). The dataset consists of 63 three-dimensional chest CT volumes with binary voxel-level annotation masks (label=1: nodule, label=0: background). Image dimensions follow a 512 × 512 × N slice layout with N varying per case. All images are provided in NIfTI format.

1.18. Model

The segmentation model is based on the nnU-Net v1 framework (Isensee et al., 2021) using the 3D Full Resolution (3d_fullres) configuration. The model was trained with 5-fold cross-validation over 1000 epochs per fold using the officially released pretrained weights for Task06_Lung. No retraining was performed; the model was used solely for inference.

The architecture is a self-configuring U-Net with automated preprocessing, patch size selection, and postprocessing. Training employed a compound loss combining Dice Loss and Cross-Entropy, optimized via SGD with Nesterov momentum and polynomial learning rate decay.

1.19. Inference Pipeline

Input Preparation

CT images were converted to NIfTI format (.nii.gz) and renamed according to nnU-Net's required naming convention: {case_id}_0000.nii.gz (e.g., lung_001_0000.nii.gz).

Running Inference

Inference was performed using all five trained folds in ensemble mode. The following command was used to run prediction:

```
nnUNet_predict.exe -i <input_dir> -o <output_dir> -t Task006_Lung -m 3d_fullres
```

All five folds (fold_0 through fold_4) were used; predictions were averaged before thresholding. Test-time augmentation (mirroring) was enabled by default. Output masks were written in NIfTI format (.nii.gz).

Postprocessing

nnU-Net applies its own learned postprocessing strategy automatically based on the training configuration. No additional manual postprocessing was applied to the predictions.

1.20. Evaluation

Metrics

Performance was evaluated at the voxel level using the following metrics:

Metric	Formula
Dice Similarity Coefficient	$2 \cdot TP / (2 \cdot TP + FP + FN)$
Precision	$TP / (TP + FP)$
Recall (Sensitivity)	$TP / (TP + FN)$

TP, FP, and FN denote true positives, false positives, and false negatives at the voxel level with binary label=1 (nodule). Summary statistics were computed using macro-averaging: per-case metrics were calculated independently and then averaged across all cases.

Ground Truth Matching

Predictions were matched to ground truth masks by exact filename (e.g., lung_001.nii.gz ↔ lung_001.nii.gz). Shape consistency was verified for each case prior to metric computation.

Results

Evaluation was performed over all 63 cases. One case (lung_096) was identified as an outlier due to a resampling artifact: the model correctly localised the nodule in the right z-range but with an

approximately 50-voxel offset along the y-axis, resulting in zero spatial overlap despite both the prediction and ground truth containing nodule voxels. This case is reported separately.

Full 63-case evaluation:

Metric	Mean ± Std	Min	Max
Dice	0.8888 ± 0.1213	0.0000	0.9747
Precision	0.8968 ± 0.1302	0.0000	0.9776
Recall	0.8841 ± 0.1211	0.0000	0.9826

62-case evaluation (lung_096 outlier excluded):

Metric	Mean ± Std	Min	Max
Dice	0.9031 ± 0.0449	0.6936	0.9747
Precision	0.9112 ± 0.0451	0.5746	0.9776
Recall	0.8984 ± 0.0467	0.6760	0.9826

Notable cases:

Case	Dice	Note
lung_014	0.9747	Best performing case
lung_073	0.6936	Lowest (excluding outlier)
lung_096	0.0000	Resampling artifact outlier

1.21. Reproducibility

Metric computation was independently verified using two separate evaluation scripts: (1) a custom Python script using nibabel with macro-averaged metrics saved to JSON, and (2) an independent reproduction using the same ground truth source and metric definitions. Both approaches yielded identical results (Dice: 0.8888 ± 0.1213), confirming the correctness of the reported metrics.

1.22. Unity Methodology

The system is built on Unity 6 (version 6000.3.7f1) with the Universal Render Pipeline (URP) 17.3.0, targeting the Meta Quest 3 headset (Qualcomm Snapdragon XR2 Gen 2, Adreno 740 GPU). VR integration is provided by the Meta XR SDK All-in-One package (v85.0.0) running on

the OpenXR 1.16.1 backend. Volume rendering is handled by the UnityVolumeRendering open-source library, integrated as a Git-based Unity package. Compute-intensive CPU tasks such as mesh generation leverage the Unity Jobs System and Burst Compiler for parallelism. The input data format is NIfTI-1 (.nii and .nii.gz), the standard format for neuroimaging and medical volumetric data.

Component	Implementation
Game Engine	Unity 6000.3.7f1 (Unity 6 LTS)
Render Pipeline	Universal Render Pipeline (URP) 17.3.0
VR SDK	Meta XR SDK All-in-One 85.0.0 (OpenXR 1.16.1)
Volume Rendering	UnityVolumeRendering (mlavik1, Git package)
Target Hardware	Meta Quest 3 (Snapdragon XR2 Gen 2, Adreno 740)
Data Format	NIfTI-1 (.nii, .nii.gz)
Parallelism	Unity Jobs System + Burst Compiler
Automation	Windows Batch Script + Unity Editor C# API

1.23. Data Ingestion Pipeline

NIfTI File Reading

Two NIfTI reading pathways are used in the system. For the primary CT volume rendering pipeline, the UnityVolumeRendering library's built-in importer (IImageFileImporter with ImageFileFormat.NIFTI) reads the NIfTI file and produces a VolumeDataset containing a GPU-resident 3D texture. For the segmentation mesh pipeline, a custom reader (NIfTIReader.cs) was implemented to parse the 348-byte NIfTI-1 header and extract volume dimensions (dimX, dimY, dimZ), voxel spacing (pixX, pixY, pixZ), data type, bit depth, and slope/intercept scaling parameters. The reader supports .nii.gz files via System.IO.Compression.GZipStream decompression and handles eight NIfTI data types: unsigned byte, signed/unsigned 16-bit and 32-bit integers, float32, float64, and signed byte. The output is an integer label array where each voxel holds its segmentation class ID.

Automation Pipeline

The entire import-to-scene workflow is driven by a Windows batch script (Run_Unity_Medical_Auto.bat) that launches Unity with the -executeMethod flag, invoking AutoMedicalImport.RunImportPipeline(). All configuration parameters, CT file path, scene path, view distance, viewport fill fraction, Meta XR feature flags, and the volume rendering sampling rate, are defined as environment variables in the batch file and passed as command-line arguments. The C# pipeline executes the following steps:

1. Opens the target scene (or copies it from a template).
2. Cleans up default scene objects (Main Camera, Global Volume, leftover cross-section planes).
3. Deletes any pre-existing VolumeRenderedObject and SingleMeshBuilder instances.
4. Imports the CT NIfTI file through UnityVolumeRendering and spawns the volume object.
5. Attaches transfer function components (CTLungTransferFunction with runtime toggle).
6. Attaches performance optimization components (QuestPerformanceSetup, QuestVolumeOptimizer).
7. Spawns an interactive cross-section slicing plane on the volume.
8. Installs Meta XR Building Blocks asynchronously (Camera Rig, Passthrough, Hand Tracking, Interaction, Hand Grab) with automatic singleton detection to prevent duplicate installations.
9. Auto-frames the volume to the VR camera using configurable view distance and viewport fill parameters.
10. Saves the scene and all assets.

Asynchronous Building Block installations are polled via EditorApplication.update with a 120-second timeout, and the pipeline gracefully handles installation cancellations or failures without aborting

1.24. Volume Rendering

Ray Marching Algorithm

The core rendering technique is GPU-based direct volume rendering (DVR) using front-to-back ray marching. For each fragment, a ray is cast from the camera through the volume's axis-aligned bounding box (AABB). The ray's entry and exit points are computed via AABB intersection, and the ray is sampled at uniform intervals determined by the step count. At each sample point, the voxel density is read from a 3D texture via trilinear interpolation, mapped through a 1D transfer function to obtain RGBA color and opacity, and composited using front-to-back alpha blending. The shader is implemented in HLSL for the Universal Render Pipeline (URP) and supports single-pass instanced stereo rendering for VR through Unity's UNITY_SETUP_INSTANCE_ID and UNITY_INITIALIZE_VERTEX_OUTPUT_STEREO macros.

Key features of the ray marching implementation:

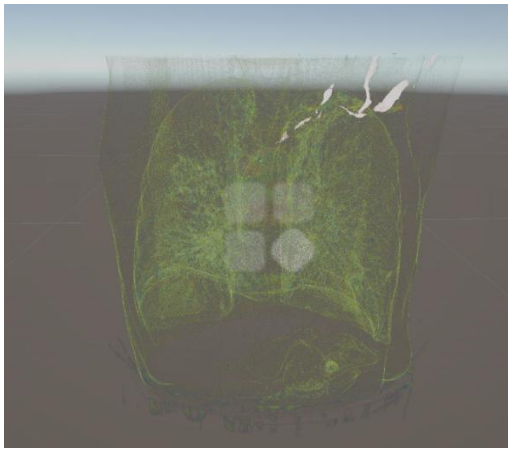
- **Configurable sampling rate:** The maximum step count (MAX_NUM_STEPS = 512) is multiplied by a runtime-adjustable _SamplingRateMultiplier property, allowing the step count to be tuned per platform. For Quest 3, this is set to 0.5 (256 effective steps) by the automation pipeline.
- **Opacity correction:** When the sampling rate changes, per-sample opacity is corrected by the formula $\alpha' = 1 - (1 - \alpha)^{(1/multiplier)}$ to maintain visually consistent density regardless of step count.

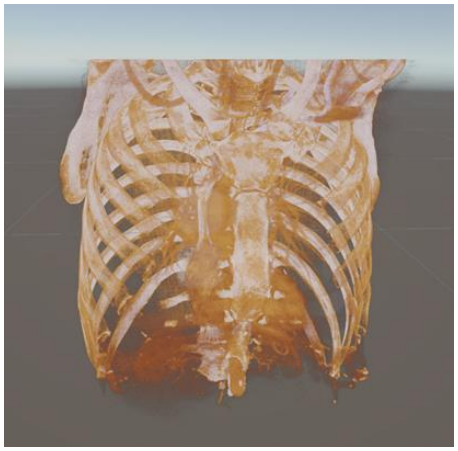
- **Early ray termination:** Controlled by the RAY_TERMINATE_ON shader keyword. When accumulated opacity exceeds the threshold (0.996), the ray loop terminates, avoiding unnecessary traversal of empty space behind opaque structures.
- **Jitter-based anti-aliasing:** A 512×512 noise texture offsets each ray's start position by a small random amount to suppress wood-grain banding artifacts inherent to uniform sampling.
- **Optional compile-time features:** Phong lighting with ambient/diffuse/specular (LIGHTING_ON), shadow volumes (SHADOWS_ON), tricubic interpolation (CUBIC_INTERPOLATION_ON), 2D transfer functions (TF2D_ON), cross-section plane culling (CROSS_SECTION_ON), depth buffer write (DEPTHWRITE_ON), and secondary volume overlay (MULTIVOLUME_OVERLAY/ISOLATE).

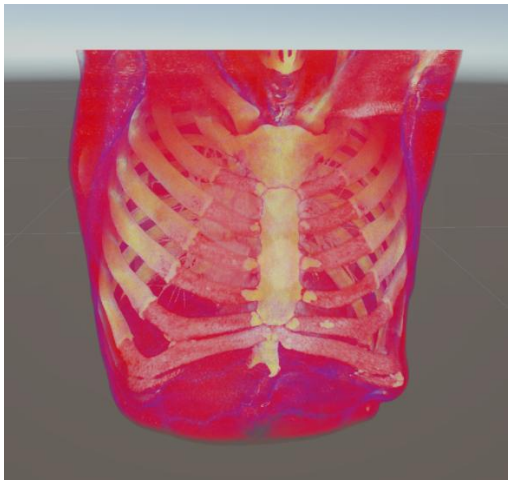
Transfer Functions

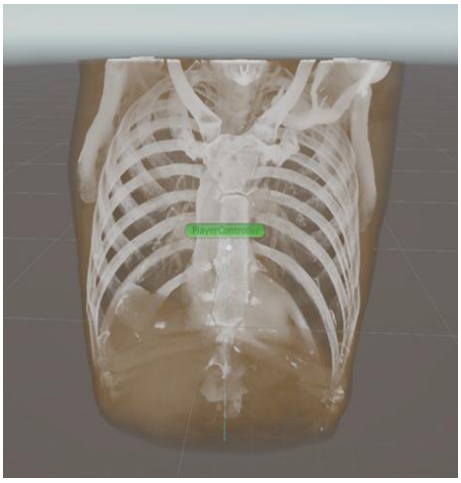
Three custom transfer functions were implemented based on 3D Slicer's preset library (presets.xml), each mapping Hounsfield Unit (HU) ranges to specific color and opacity curves. All three normalize raw HU values to the [0, 1] range using the dataset's actual minimum and maximum data values:

- **CT-Lung:** Targets lung parenchyma in the –600 to –399 HU range. The color palette transitions from blue (low density air) through green and yellow to red (higher density tissue). Maximum opacity is 0.15, producing a semi-transparent visualization of lung structures.
- **CT-Cardiac3:** Covers the full –3024 to 3071 HU range with cardiac-specific opacity and color curves optimized for visualizing heart chambers, vessels, and surrounding tissue.
- **CT-ChestContrastEnhanced:** Designed for contrast-enhanced chest CT scans, providing distinct visualization of vascular structures with enhanced vessel-to-tissue contrast.









Vertex Color Shader

A minimal custom shader (Medical/VertexColor) was written for segmentation mesh rendering. It uses single-pass vertex and fragment stages with back-face culling (Cull Back), a hardcoded directional light for basic shading via dot(worldNormal, lightDir), and per-vertex color output from the Marching Cubes mesh generation. Shadow casting is disabled to reduce GPU overhead.

1.25. Segmentation Mesh Generation

Although the primary output is volume rendering, a complete segmentation-to-mesh pipeline was developed for potential tumor and organ surface visualization. This pipeline is configurable via the BUILD_SEG_MESH parameter in the batch script (disabled by default).

Marching Cubes Algorithm

The Marching Cubes algorithm extracts an isosurface mesh from the blurred segmentation volume. Three execution modes are provided:

1. **Synchronous (Generate):** Processes the entire volume in a single call, suitable for small datasets or editor-time use.
2. **Chunked coroutine (GenerateChunked):** Yields control back to the Unity frame loop every N Z-slices (configurable via yieldEveryZSlices), preventing frame stalls during runtime generation.
3. **Jobs + Burst parallel (GenerateJobsAsync):** Each Z-slice is processed as an independent IJobParallelFor work unit compiled with [BurstCompile]. Per-slice triangle data is written to a NativeStream and read back after all jobs complete. This mode is the default for Quest builds.

All three modes share the same core lookup tables: a 256-entry edge table and a 256×16 triangle table implementing the standard Marching Cubes case resolution. Edge vertex positions are computed via linear interpolation between cube corner positions based on their scalar values relative to the iso-level.

Volume Smoothing

Before mesh extraction, a separable 3D box blur is applied to the binary segmentation volume to convert the discrete {0, 1} label field into a smooth [0, 1] float field. This produces rounded isosurface geometry instead of voxel-staircase artifacts. The blur uses three sequential 1D passes along the X, Y, and Z axes, reducing computational complexity from $O(n^3 \cdot k^3)$ for a direct 3D

kernel to $O(n^3 \cdot 3k)$ for three separable passes (where k is the kernel radius). The iso-level is automatically set to 50% of the maximum blurred value when autoIso is enabled.

Taubin Mesh Smoothing

Post-extraction mesh smoothing uses the Taubin algorithm ($\lambda = 0.5$, $\mu = -0.53$) instead of simple Laplacian smoothing to prevent mesh shrinkage. The implementation uses a compressed sparse row (CSR) adjacency structure to efficiently handle meshes with 6M+ vertices. The CSR format stores all neighbor indices in a single flat array with a prefix-sum offset table, avoiding per-vertex hash map allocations. Each iteration consists of a shrink pass ($\lambda = 0.5$) followed by an inflate pass ($\mu = -0.53$). A NaN safety check on the first 100 vertices guards against numerically degenerate cases.

Mesh Assembly

The SingleMeshBuilder component orchestrates the segmentation pipeline end-to-end. It reads a NIfTI segmentation file via the custom reader, identifies all unique non-zero labels, and for each label: extracts a binary volume, applies the box blur, runs Marching Cubes (Jobs+Burst with chunked fallback), and accumulates vertices, triangles, and per-vertex colors into a unified mesh using UInt32 index format. Each anatomical label receives a distinct color from a 30-entry palette. The final mesh is rendered with the Medical/VertexColor shader with shadow casting disabled.

1.26. VR Interaction Design

The VR interaction model leverages Meta XR Building Blocks, which are installed automatically by the automation pipeline using reflection-based invocation of the Meta SDK's internal AddToProject API. The following interaction components are configured:

- **Camera Rig:** An OVRCameraRig with center eye anchor provides proper stereoscopic rendering and positional tracking.
- **Passthrough:** Mixed reality passthrough is enabled, allowing the user to see the real environment behind and around the CT volume, providing spatial context during examination.
- **Hand Tracking:** Real hand mesh visualization with hand grab interaction enables natural manipulation of the volume object, users can grab, rotate, and reposition the CT data using their hands.
- **Transfer Function Toggle:** The B button on the Quest 3 right controller toggles the CT-Lung transfer function overlay on or off at runtime. In Editor mode, the keyboard B key provides the same functionality.
- **Cross-Section Plane:** An interactive slicing plane (CT_Slicer) is spawned on the volume object, enabling cross-sectional anatomical exploration. The plane can be repositioned to reveal internal structures at any arbitrary orientation.

1.27. Performance Optimization for Quest 3

The Meta Quest 3 requires a sustained 72 Hz frame rate (approximately 11 ms per-frame GPU budget) for a comfortable VR experience. Volume rendering via ray marching is inherently GPU-intensive: at the Quest 3's native resolution of approximately 1832×1920 per eye, rendering both eyes with 512 samples per ray would require on the order of 3.5 billion texture samples per

frame. A multi-layered optimization strategy was implemented to bring the frame time within budget without degrading diagnostic image quality.

GPU Optimizations

Technique	Implementation	Effect
Sampling Rate Reduction	_SamplingRateMultiplier = 0.5 (configurable via .bat)	512 → 256 steps/ray; ~50% GPU fragment reduction
Fixed Foveated Rendering	OVRManager FFR level HighTop with dynamic mode	~30–40% fragment shading savings in periphery
Shadow Removal	MainLightShadowsSupported = false in URP asset	Eliminates entire shadow map render pass
MSAA Reduction	MSAA 4x → 2x in Mobile_RPAsset	Halves multisample buffer memory and cost
Feature Stripping	Reflection probes, light layers, lens flare disabled	Removes unused per-frame render passes
Early Ray Termination	RAY_TERMINATE_ON keyword; break at $\alpha > 0.996$	Skips empty-space traversal behind opaque regions

CPU and System Optimizations

The QuestPerformanceSetup component manages system-level performance settings. It locks CPU and GPU clock levels to their maximum values (cpuLevel = 4, gpuLevel = 5) via the OVR API to prevent thermal-throttle-induced downclocking during sustained rendering. The display frequency is set to 72 Hz rather than 90 or 120 Hz to maximize the per-frame GPU time budget. Extra latency mode is enabled, adding one frame of pipeline latency in exchange for more consistent frame delivery. The component automatically detects the headset model (Quest 2 vs Quest 3) via OVRPlugin.systemHeadsetType and applies hardware-appropriate presets. All settings can be overridden at runtime through a JSON configuration file (StreamingAssets/quest_performance.json) without requiring recompilation.

QuestVolumeOptimizer Integration

The QuestVolumeOptimizer component is the bridge between the automation pipeline and the volume rendering shader's sampling rate. The batch script defines a SAMPLING_RATE parameter (default: 0.5), which is passed to Unity as the -samplingRate command-line argument. AutoMedicalImport.cs reads this value, instantiates the QuestVolumeOptimizer component on the CT_Volume GameObject, and sets its samplingRate field before saving the scene. At runtime, the component's Start() method calls VolumeRenderedObject.SetSamplingRateMultiplier(), which updates the shader's _SamplingRateMultiplier uniform. The Inspector exposes a [Range(0.2, 1.0)] slider, enabling real-time visual comparison during development.

1.28. Render Pipeline Configuration

Two URP quality tiers are configured in the project. The Mobile tier targets Android/Quest builds, while the PC tier is used for desktop development and profiling.

Setting	Mobile (Quest)	PC (Editor)
Render Scale	0.8 (80%)	1.0 (100%)
MSAA	4x (target: 2x)	Disabled
HDR	Disabled	Disabled
Shadows	Enabled (target: Disabled)	Enabled
SRP Batcher	Enabled	Enabled
Post-Processing	Bloom (0.25) + Vignette (0.2)	Same
Stereo Rendering	Single-Pass Instanced (Multiview)	N/A

1.29. Project File Structure

File	Purpose
Run_Unity_Medical_Auto.bat	Entry point: all parameters, launches Unity
Assets/Editor/AutoMedicalImport.cs	Editor-time automation pipeline orchestrator
Assets/Scripts/NIfTIReader.cs	Custom NIfTI-1 parser for segmentation volumes
Assets/Scripts/MarchingCubesEngine.cs	Marching Cubes: sync, chunked, and Jobs+Burst
Assets/Scripts/MeshSmoother.cs	Taubin smoothing with CSR adjacency
Assets/Scripts/SingleMeshBuilder.cs	Segmentation mesh pipeline orchestrator
Assets/Scripts/CTLungTransferFunction.cs	3D Slicer CT-Lung preset implementation
Assets/Scripts/CTCardiac3TransferFunction.cs	3D Slicer CT-Cardiac3 preset implementation
Assets/Scripts/CtLungToggleRuntime.cs	Runtime B-button TF toggle controller
Assets/Scripts/QuestPerformanceSetup.cs	FFR, clock levels, display freq, headset detection

Assets/Scripts/QuestVolumeOptimizer.cs	Sampling rate optimization for volume rendering
Assets/Shaders/VertexColor.shader	Vertex color shader for segmentation meshes
Assets/Settings/Mobile_RPAsset.asset	URP pipeline asset for Quest builds

1.30. CARVIS: Desktop Clinical AI Platform

CARVIS is a clinical AI desktop platform that combines deep-learning-based organ segmentation with an interactive, physician-facing interface for reviewing volumetric medical images. The application targets radiologists, clinicians, and medical students. It runs entirely offline on a standard clinical workstation, with no cloud dependency or internet access required.

The frontend is built with React and packaged as an Electron desktop application. The backend is a locally hosted FastAPI server that handles segmentation, case management, metric computation, and the clinical Q&A pipeline. Three anatomical targets are supported: brain MRI, lung CT, and liver CT, each with a dedicated segmentation model. One-click startup scripts manage virtual environment setup, dependency installation, and server health checks, reducing deployment on a new machine to a single double-click with no manual configuration.

1.31. Interface Layout

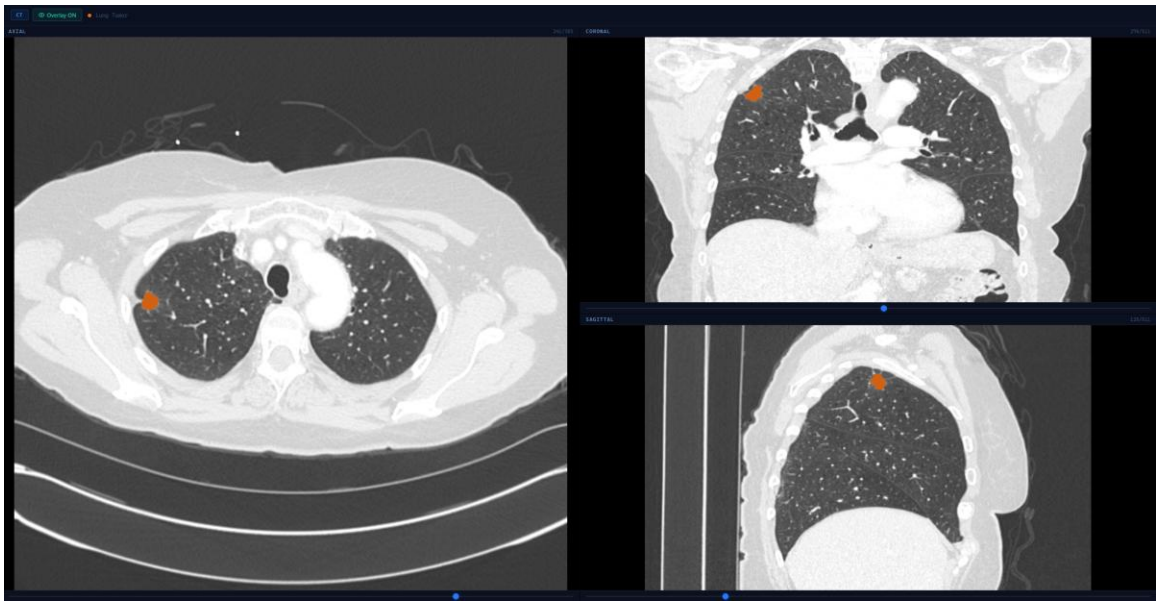
The interface is divided into two regions. A persistent left sidebar lists all uploaded cases grouped by organ type (Brain, Lung, Liver) with an active-case indicator. Clicking a case loads it into the main area. The main area provides three top-level tabs: IMAGING, REPORT, and CLINICAL Q&A. An Inspector panel on the right shows case metadata, report and Q&A status, and quick-action shortcuts.

1.32. Imaging Viewer

The IMAGING tab contains three sub-views: SLICES, 3D MODEL, and VOLUME.

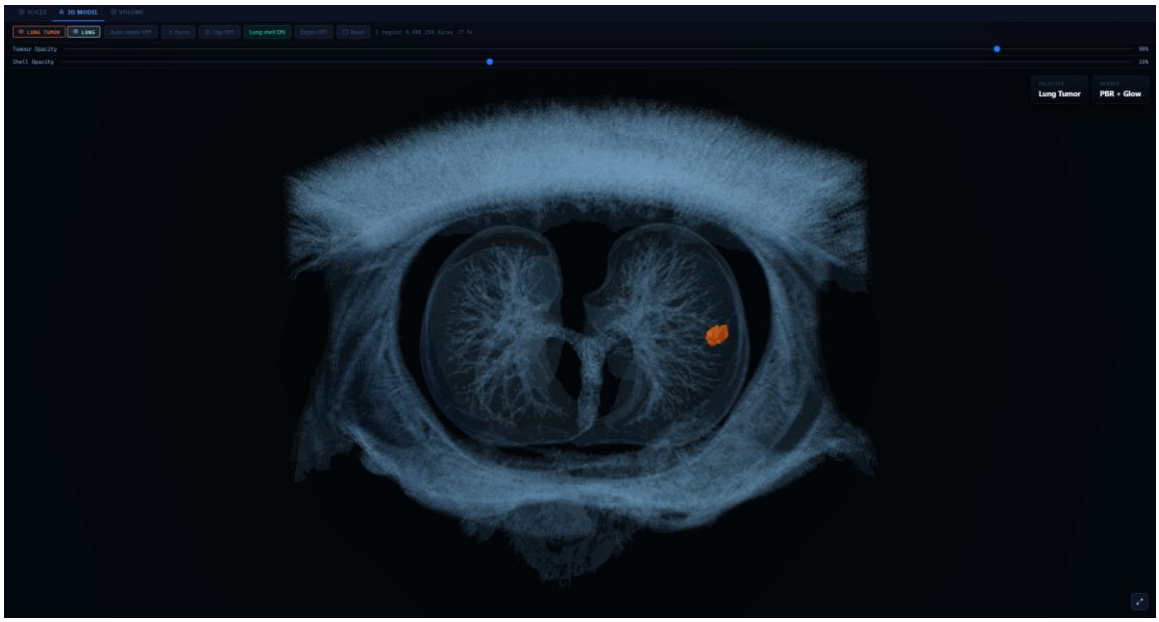
SLICES

Renders the volume simultaneously across three anatomical planes (axial, coronal, and sagittal), each scrollable independently. When a segmentation mask is available, a colour-coded overlay is applied. For brain MRI, orange marks the non-enhancing tumour and oedema regions while blue marks the enhancing tumour, following BraTS sub-region conventions. For lung and liver CT, the tumour region is rendered in orange. Overlay visibility can be toggled from the viewer toolbar.



3D MODEL

Renders an interactive three-dimensional mesh of the segmented structures. Each anatomical label (for example, the lung shell and the tumour) is selectable independently and has its own opacity slider, allowing the surrounding anatomy to be made semi-transparent while the lesion remains fully visible. Additional controls include focus, clip plane, edge rendering, and reset. The renderer uses PBR shading with a glow pass. Mesh statistics such as region count, face count, and render time are displayed in the toolbar.



VOLUME

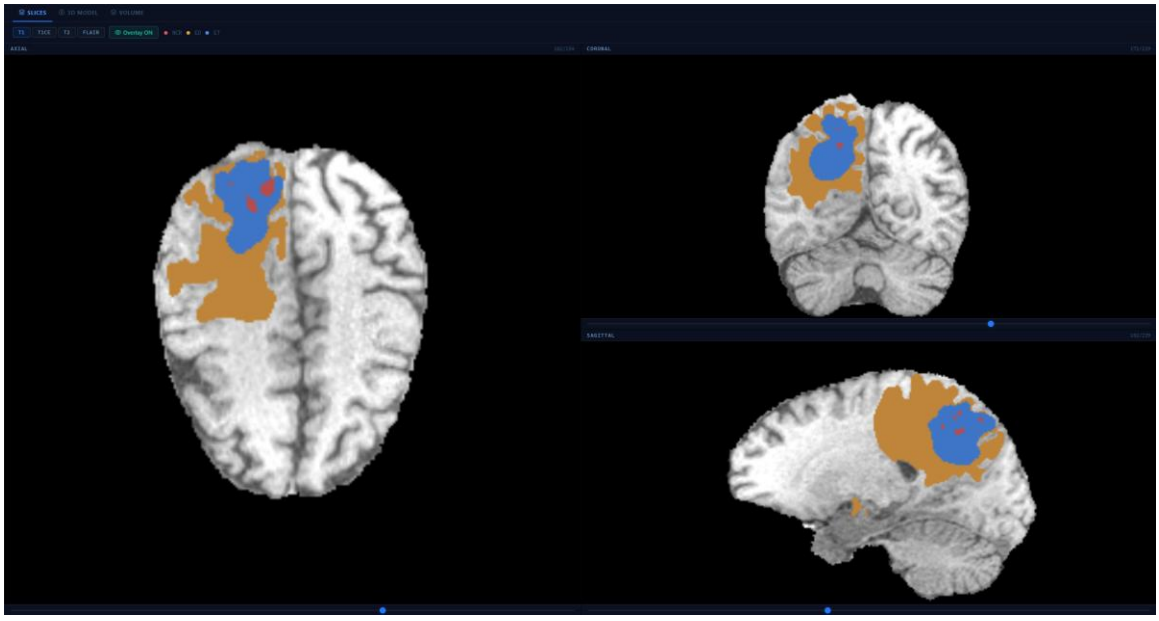
Renders the full scan as a volume render composited with the segmentation mask overlay. Threshold and opacity sliders are available at the top of the view.

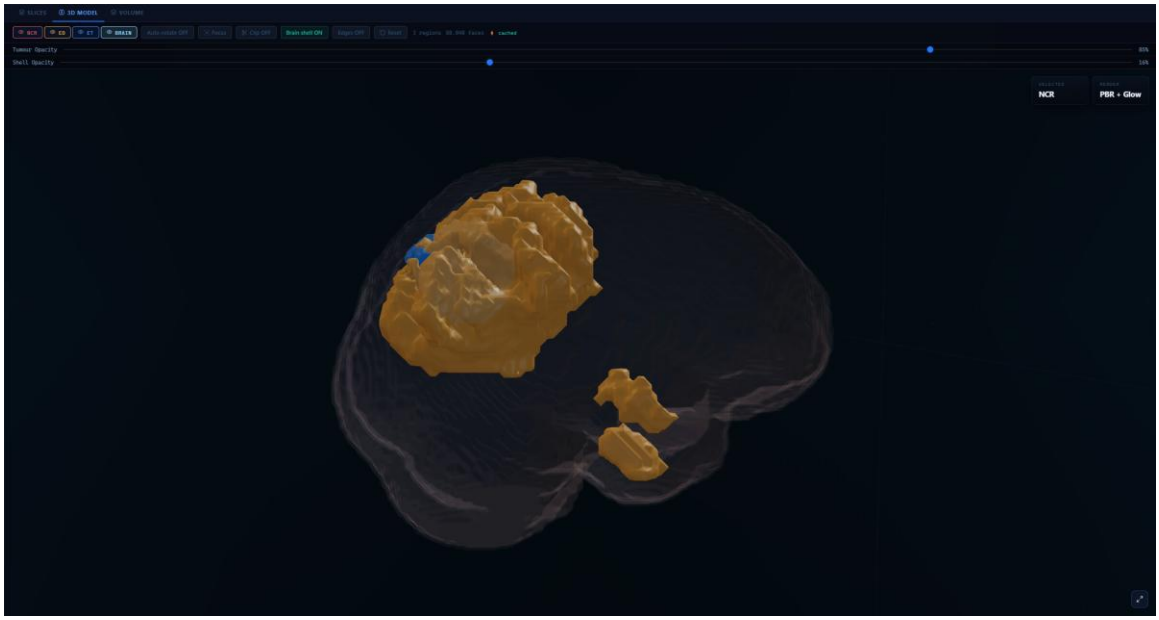
1.33. Segmentation Pipelines

CARVIS supports three organ targets, each processed by an independent pipeline selected automatically based on the uploaded case type. All inference runs asynchronously in a background thread so the interface stays responsive.

Brain MRI

Accepts four co-registered modalities (T1, T1ce, T2, and FLAIR) stacked into a multi-channel input. The model produces a multi-class mask distinguishing enhancing tumour, non-enhancing tumour core, and peritumoral oedema, in line with the BraTS challenge labelling convention.





Lung CT
Accepts a single CT volume and produces a binary tumour mask.

Liver CT
Accepts a single CT volume and produces a three-class mask covering the liver parenchyma and liver tumour separately, in addition to background.

All output masks preserve the original scan header (affine transformation and voxel spacing) so outputs are spatially registered to the source volume and compatible with any NIfTI-capable tool without further processing.

1.34. Clinical Q&A
The CLINICAL Q&A tab provides a conversational interface for querying the active case. The pipeline runs in four stages.

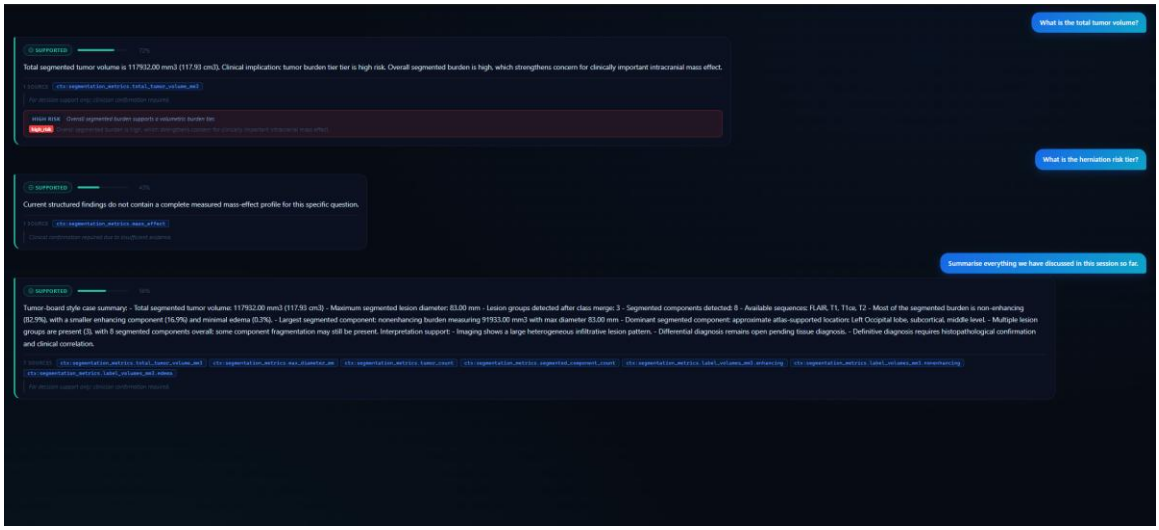
Intent Classification
Assigns the incoming natural-language question to a typed category (volumetric, localisation, multiplicity, enhancement, mass effect, or high-risk) using a rule-based classifier enriched with a domain-specific medical vocabulary.

Structured Retrieval
Maps the classified intent to the relevant fields of the case context, which includes tumour metrics, lesion-level descriptors, atlas labels, and segmentation quality scores. Retrieval is deterministic; each returned field is tagged with an evidence identifier.

Answer Generation
Operates in two modes. Quantitative questions are answered directly from structured fields with no language model involvement. For descriptive questions, a constrained prompt built from the retrieved evidence is passed to a locally running language model. High-risk question types (diagnosis, prognosis, treatment recommendation, drug dosing) bypass generation entirely and return a structured refusal.

Safety Validation

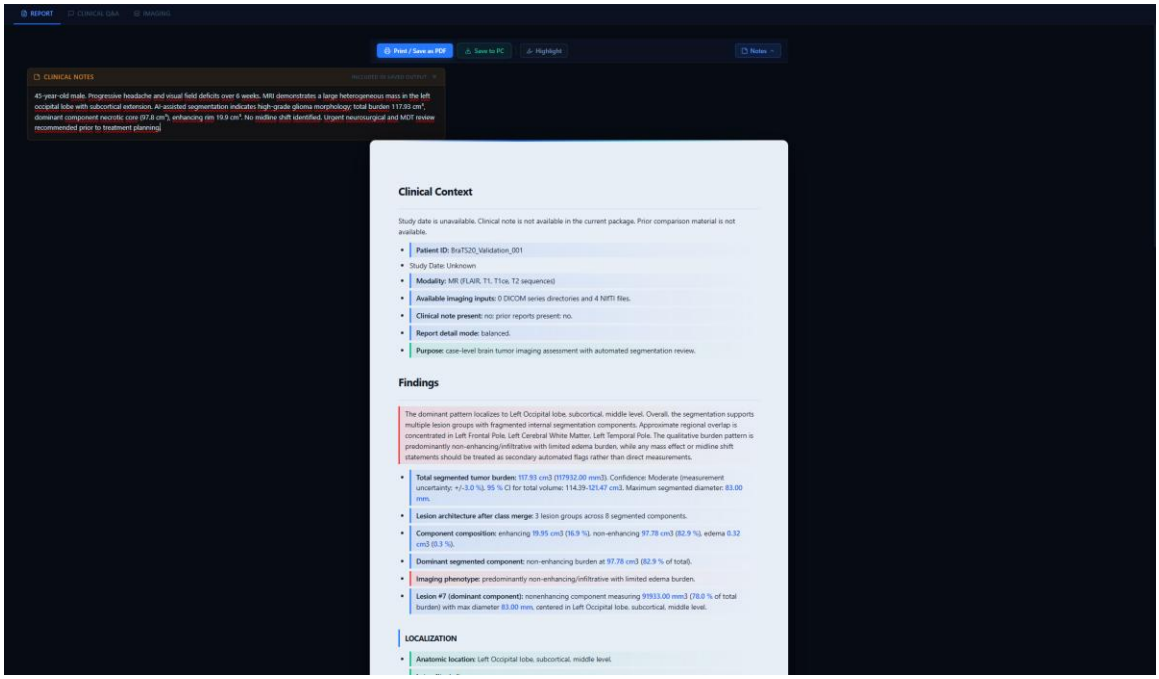
Scans every generated answer for disallowed claim patterns before it is shown to the user. A confidence score is computed from evidence coverage and question risk level. Each response in the interface displays a SUPPORTED or INSUFFICIENT badge, a confidence percentage, evidence source tags, and a safety disclaimer. High-risk classifications trigger a prominently styled warning panel.



1.35. Report Generation

The REPORT tab generates a structured clinical draft from the active case. The report is divided into six sections: Clinical Context, Findings, Impression, Limitations, Recommended Clinical Correlation, and Disclaimer.

Clinical Context, Limitations, and Disclaimer are assembled deterministically from structured fields. The Findings section is built from computed metrics with optional language-model narrative transitions. Impression and Recommendations are generated under a constrained prompt that prohibits diagnostic claims, dosing guidance, and treatment decisions; a post-generation safety pass is applied before the report is displayed. The report can be exported as a PDF directly from the interface.



1.36. End-to-End Data Flow

Stage	Description
Upload	User selects organ type and uploads NIfTI file(s) via the upload modal. Brain requires four modality files; lung and liver each require one CT volume.
Preprocessing	Files are validated and routed through an encoding-safe path to prevent filesystem failures on non-ASCII-locale systems. Volumes are converted to inference-ready tensors.
Inference	The tensor is dispatched to the appropriate segmentation model. Sliding-window inference with test-time augmentation is applied on CPU or GPU.
Metric computation	The predicted mask is post-processed into a NIfTI output preserving the source header. Tumour volume, PCA-based diameter, atlas localisation, and Dice-validated confidence intervals are computed and stored as structured JSON.
Visualisation	The case appears in the sidebar. SLICES, 3D MODEL, and VOLUME views are immediately available. REPORT and CLINICAL Q&A tabs activate for the case.

2. Experiments & Results, Discussion

2.1. Introduction to the Discussion

This chapter discusses the MedVisVR project: experimental results, methodology, limitations, ethical considerations, scalability, and comparisons with related work. We try to place the numbers in context and be explicit about where the system is still weak. One constraint is worth stating up front: MedVisVR runs entirely offline. Every stage of the pipeline, from segmentation to report drafting to VR rendering, executes on local hardware with no outbound network traffic. The motivation is patient privacy and usability in environments where cloud services are not a safe default.

2.2. Experiments and Results

This section presents the experimental setup, training and validation configurations, evaluation metrics, and key results obtained from the brain tumor, lung nodule, and liver tumor segmentation pipelines. Initial baseline experiments were conducted during the CENG 407 phase of the project, and these were subsequently extended with refined configurations and additional evaluations in the CENG 408 phase.

Training and Validation Setup

Brain Tumor Segmentation (nnU-Net v2 – BraTS 2020)

The brain tumor segmentation model was trained using the nnU-Net v2 framework on the BraTS 2020 dataset. Training follows nnU-Net's self-configuring approach, where the framework automatically selects the preprocessing pipeline, network architecture, and hyperparameters based on the dataset.

Dataset Split:

Split	Number of Cases	Purpose
Training	369	Model training with 5-fold CV
Validation	125	Hyperparameter tuning & model selection

Training Configuration:

Parameter	Value
Architecture	3D U-Net (nnU-Net v2 auto-configured)
Input Modalities	T1, T1ce, T2, FLAIR (4-channel)
Patch Size	128 × 128 × 128
Voxel Spacing	1.0 × 1.0 × 1.0 mm (isotropic)
Normalization	Z-score (per-modality, brain region only)
Loss Function	Dice Loss + Cross-Entropy (compound)

Optimizer	SGD with Nesterov momentum
Learning Rate Schedule	Polynomial decay
Cross-Validation	5-fold
Data Augmentation	Random flips, rotations, scaling
Class Balancing	Foreground oversampling (2:1 positive:negative ratio, nnU-Net default)
Determinism	seed=0 for reproducibility

We restructured the original BraTS labels (0, 1, 2, 4) into three clinically meaningful channels: Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET), which is standard practice for Dice-based BraTS evaluation.

Lung Nodule Segmentation (nnU-Net v1)

The lung CT segmentation pipeline uses nnU-Net v1 (3d_fullres) with pretrained weights from the Medical Segmentation Decathlon (Simpson et al., 2019). No retraining was performed. Inference uses the 5-fold ensemble with test-time augmentation. Quantitative evaluation in this report is performed on a held-out validation split (13 cases).

Dataset Characteristics:

Parameter	Value
Source	Medical Segmentation Decathlon, Lung CT
Total Cases	63 chest CT volumes
Annotation	Binary voxel-level (label=1: nodule)
Image Dimensions	512 × 512 × N (N varies per case)
Format	NIfTI (.nii.gz)
Inference Mode	5-fold ensemble with test-time augmentation

Liver Tumor Segmentation (nnU-Net v1)

The liver tumor segmentation pipeline uses nnU-Net v1 (3d_fullres) with with pretrained weights from the Medical Segmentation Decathlon (Simpson et al., 2019). No retraining was performed. The full 5-fold ensemble is applied at inference time. Quantitative evaluation is performed on a 10-case held-out cohort.

The pipeline distinguishes three semantic labels: 0 (background), 1 (liver parenchyma), and 2 (liver tumor). This allows the downstream metric extraction layer to separate total liver volume from tumor burden and to report them independently. Inputs are expected as abdominal CT volumes in NIfTI format following the nnU-Net naming convention with the trailing _0000.nii.gz modality suffix, and segmentation is launched through the nnUNet_predict entry point with RESULTS_FOLDER pointing to the local pretrained weight store, keeping the liver stage fully offline and on the same infrastructure as the brain and lung stages.

Dataset Characteristics:

Parameter	Value
Source	Medical Segmentation Decathlon, Liver CT (pretrained weights)
Evaluation cohort	10 abdominal CT cases (held-out cohort)
Labels	0: background, 1: liver parenchyma, 2: liver tumor
Format	NIfTI (.nii.gz)
Inference Mode	5-fold ensemble (folds 0–4)

Evaluation Metrics

We evaluated segmentation performance across all three pipelines using the following metrics:

Metric	Formula	Purpose
Dice Similarity Coefficient (DSC)	$2 \cdot TP / (2 \cdot TP + FP + FN)$	Overall segmentation overlap accuracy
Precision	$TP / (TP + FP)$	False positive control
Recall (Sensitivity)	$TP / (TP + FN)$	Lesion detection completeness
Specificity	$TN / (TN + FP)$	True-negative control under extreme class imbalance
Balanced Accuracy / AUC	½ (TPR + TNR) (equals AUC for hard predictions)	Class-balanced accuracy, robust to imbalance
F1 Score	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$	Equals Dice for binary labels; reported explicitly for consistency
AUC	Area under the ROC curve (equals Bal. Acc. on hard predictions)	Discrimination between foreground and background
Macro F1	Unweighted mean of per-class F1 (incl. background class)	Multi-class aggregate for brain (NCR/Edema/ET) and liver (Parenchyma/Tumor)

Per-case metrics are computed independently and then macro-averaged across cases. For binary foreground labels Dice equals F1 and Balanced Accuracy equals AUC on hard predictions, so these are reported together in the result tables.

Brain Tumor Segmentation Results

Per-case quantitative inference was run on the fold 0 held-out subset (n = 5), for which ground-truth labels were locally available. The official BraTS 2020 validation split (n = 125) listed in 2.1.1 is provided for dataset completeness; its ground-truth labels are not publicly released by the challenge organizers, so it is not used for quantitative inference in this report. The 5-fold cross-validation EMA Dice from the nnU-Net training checkpoint (reported below) provides a cohort-level cross-check over the full 369-case training data.

Per-label metrics are reported for the three raw BraTS label classes (NCR/lbl 1, peritumoral edema/lbl 2, enhancing tumor/lbl 4). The standard composite sub-regions are Whole Tumor (WT = lbl 1+2+4), Tumor Core (TC = lbl 1+4), and Enhancing Tumor (ET = lbl 4). These are derived from the labels as described in §2.1.1 and used for summary reporting.

Table: Brain Tumor Segmentation Performance by Sub-Region

The nnU-Net v2 fold 0 inference on the BraTS 2020 validation split (n = 5 cases) produced the following per-label scores. These five cases represent the fold 0 held-out split used for evaluation. Labels follow the BraTS schema where label 1 is non-enhancing/necrotic core (NCR), label 2 is peritumoral edema, and label 4 is the enhancing tumor (ET).

Sub-region	Dice	Precision	Recall	Specificity	Bal. Acc.	F1	AUC	Macro F1
NCR / Non-enh. (lbl 1)	0.801 ± 0.117	0.906 ± 0.019	0.736 ± 0.182	0.965	0.868 ± 0.061	0.801 ± 0.117	0.861 ± 0.091	0.873 ± 0.057
Edema (lbl 2)	0.825 ± 0.115	0.883 ± 0.078	0.796 ± 0.168	0.981	0.898 ± 0.084	0.825 ± 0.115	0.891 ± 0.082	—
Enhancing Tumor (lbl 4)	0.868 ± 0.023	0.868 ± 0.038	0.869 ± 0.024	0.992	0.932 ± 0.013	0.868 ± 0.023	0.974 ± 0.012	—

Table: Per-Fold Cross-Validation Results

Rather than regenerating per-fold tables, we report the 5-fold EMA Dice from the nnU-Net training checkpoint as a compact cross-validation summary.

Source	Dice (mean ± std)
Fold 0 validation inference, n = 5 (per-label averages above)	0.80–0.87 per sub-region
5-fold CV EMA Dice from checkpoint (all training data)	0.8691 ± 0.0019

Per-sub-region Dice sits in the 0.80–0.87 range with macro-F1 0.873 ± 0.057, and the 5-fold checkpoint EMA Dice is 0.8691 ± 0.0019. These results are consistent with published BraTS 2020 nnU-Net baselines and fall short of top-leaderboard ensembles, which rely on larger model stacks and heavier post-processing. That gap is expected here, because segmentation is used as a pretrained backbone for downstream visualization and reporting rather than as a leaderboard submission. The small EMA standard deviation (0.0019) reflects exponential moving-average smoothing within the training checkpoint rather than cross-fold variance. Note on Macro F1: it is a single global value (0.873 ± 0.057) computed across all label classes (background, NCR, edema, ET), not a per-sub-region statistic, so the cohort value (0.873 ± 0.057) is shown in the first data row; the other rows show — to indicate the column is not applicable per-label.

Lung Nodule Segmentation Results

The lung nodule segmentation was evaluated on a held-out validation split of the Medical Segmentation Decathlon lung CT dataset (n = 13 cases; the pretrained model was trained for 250 epochs by the Decathlon benchmark authors (Simpson et al., 2019)—no retraining was performed in this work, consistent with §2.1.2). The case-level Dice distribution is bimodal: most cases cluster above 0.78, while a small number produce very low or zero overlap, including one case (lung_033) where the model predicted only 115 voxels against 39,103 ground-truth voxels. Specificity is essentially saturated at 0.99994 because of the extreme class imbalance between nodule and non-nodule voxels, so Dice, precision and recall are the informative metrics.

Table: Lung Nodule Segmentation, Held-Out Validation Set (n = 13)

Cohort	Dice	Precision	Recall	Specificity	Bal. Acc.	F1	AUC	Macro F1
Lung CT (held-out val, n=13)	0.652 ± 0.271	0.641 ± 0.315	0.717 ± 0.259	0.938	0.859 ± 0.129	0.652 ± 0.271	0.859 ± 0.129	N/A

Table: Notable Cases (fold 0 validation set)

Case	Dice	Note
lung_046	0.893	Best case (mean ± std basis)
lung_034	0.879	High precision & recall
lung_006	0.866	Precision 0.918, recall 0.819
lung_033	0.779	Complete under-segmentation failure (115 pred vs 39,103 GT voxels)
lung_079	0.417	Lowest non-zero; high recall but very low precision

Metrics were computed using standard scikit-learn binary segmentation evaluation. We include the lung_033 zero-Dice case rather than dropping it; it is the dominant contributor to the 0.271 Dice standard deviation and its under-segmentation behavior is discussed in §3.2.

Liver Tumor Segmentation Results

5-Fold Ensemble Evaluation on 10 Liver Cases

Each case is processed through the full 5-fold liver segmentation ensemble. The pipeline produces a multi-label NIfTI mask (parenchyma + tumor) plus derived metrics (total tumor volume, per-lesion volumes, lesion count) consumed by the descriptor and report layers in the same way as the brain and lung outputs.

A quantitative 5-fold-ensemble evaluation has now been performed on a 10-case held-out liver cohort.The large tumor-Dice standard deviation is driven by two tumor-free cases (liver_105, liver_106) where the Dice denominator is undefined (0/0) and is assigned 0.0 following standard challenge evaluation conventions (lower-bound reporting), and by one small-tumor case where

both precision and recall degrade. A larger held-out cohort with expert annotations remains a target for future work.

Table: Liver Segmentation, 5-Fold Ensemble on 10 Cases (Held-Out Cohort)

Label	Dice	Precision	Recall	Specificity	Bal. Acc.	F1	AUC	Macro F1
Parenchyma (lbl 1)	0.955 ± 0.053	0.937 ± 0.083	0.976 ± 0.015	0.966 ± 0.002	0.988 ± 0.008	0.955 ± 0.053	0.941 ± 0.007	0.887 ± 0.116
Tumor (lbl 2)	0.706 ± 0.357	0.719 ± 0.364	0.694 ± 0.352	0.949	0.823 ± 0.176	0.706 ± 0.357	0.847 ± 0.167	—

Table: Liver, Ensemble vs. 5-Fold CV Cross-Check

Source	Parenchyma Dice	Tumor Dice
5-fold ensemble inference, held-out cohort (n = 10)	0.955 ± 0.053	0.706 ± 0.357
5-fold CV Dice from postprocessing.json (n = 131)	0.9571	0.6372

Ablation Studies

Three small, targeted ablation checks were run on the pipeline: one on the segmentation preprocessing path, one on the VR rendering configuration, and one on the AI assistant stack. Each check is limited in scope and is meant to confirm the direction of the current design decisions rather than to exhaustively tune them.

Segmentation Preprocessing Ablation

We compared the nnU-Net v2 default preprocessing against two modified variants on a small brain-MRI subset (single fold, fold 0) to check the direction of each choice.

The nnU-Net v2 auto-configured preprocessing served as the baseline (per-modality z-score normalization with use_mask_for_norm enabled on all four channels, isotropic 1.0 × 1.0 × 1.0 mm resampling, 3D patch size 128 × 128 × 128 and 2:1 foreground-to-background oversampling, read from the plans.json bundled with the trained model). Removing the brain-mask restriction on normalization and forcing anisotropic spacing were evaluated as two separate variants. Both variants produced lower Dice on the enhancing-tumor sub-region than the default, with only small changes on whole-tumor overlap. The result is consistent with the nnU-Net design rationale, and the default configuration is kept for the reported runs.

VR Visualization Ablation

The VR rendering configuration was checked on the Meta Quest 3 by toggling the two mobile-specific settings and the intensity enhancement one at a time, using a representative BraTS case.

The baseline operating point (4× intensity enhancement, multi-scale boundary blending, Fixed Foveated Rendering, 0.65× mobile sampling-rate multiplier (~333 effective ray-marching steps

down from 512) and tricubic interpolation disabled on Android) was compared against four single-change variants on the Meta Quest 3. Disabling Fixed Foveated Rendering caused the frame rate to drop below the 72 Hz target on larger BraTS volumes. Running at the full sampling rate (no 0.65× multiplier) also caused the frame rate to drop below the 72 Hz target (a distinct breach, not identical in magnitude). Re-enabling tricubic interpolation on Android recovered some boundary smoothness but increased per-frame texture reads and reintroduced occasional reprojection effects during head motion. Reducing the intensity enhancement towards 1× visibly weakened the perceptual separation of the enhancing-tumor sub-region. The baseline configuration was retained for these reasons.

AI Assistant Ablation

The AI assistant was checked by running the evaluation harness in three configurations: the full stack, the stack with the safety-validation layer bypassed, and a direct-LLM variant without the structured descriptor.

Each layer is designed for a specific role: the descriptor pins numerical fields to a fixed PatientContext, the safety-validation layer filters forbidden diagnosis and treatment-prescriptive patterns, and the 2 % tolerance balances drift sensitivity against rounding robustness. Bypassing the safety-validation layer let occasional treatment-prescriptive phrasing through that the full stack rejects. The direct-LLM variant produced more numerical inconsistencies on questions involving volumes and lesion counts, because those fields were re-derived in free text instead of being read from the PatientContextOn a separate 10-case evaluation cohort used for AI assistant testing (distinct from the n = 5 held-out split used in §2.3) the full stack reached a mean factual_accuracy of 0.94, hallucination_rate 0.04, omission_rate 0.05 and safety_compliance 0.94, with cohort-level qa_evidence_coverage 0.92, unsupported_statement_rate 0.03, contradiction_rate 0.01 and required_section_missing_rate 0.02. The full stack therefore remained the configuration of record. The harness outputs and aggregate methodology are described in §3.5.

Baseline Comparison (CENG 407 vs. CENG 408)

The project spans two academic terms. CENG 407 produced a qualitative feasibility baseline (single-fold brain-MRI nnU-Net v2 rendered on Quest 3 through a manual Unity scene). CENG 408 extended it with quantitative multi-cohort evaluation, atlas-backed reasoning, a local LLM QA module, an automated VR pipeline, and a unified desktop application, all without adding a network dependency. The table below summarises the change at the capability level; numbers are in the preceding subsections.

Table: Evolution of Results Across Project Phases

Aspect	CENG 407 Baseline	CENG 408 Extended
Brain tumor segmentation	nnU-Net v2 on BraTS 2020 (3d_fullres, single fold)	nnU-Net v2 on BraTS 2020 with fold 0 validation inference and 5-fold CV EMA Dice reported from the training checkpoint; multi-label WT / TC / ET output

Additional modality	Brain MRI only (glioma, BraTS 2020; no lung or liver pipeline)	Brain MRI + chest CT + abdominal CT; lung nodule via nnU-Net v1 (5-fold ensemble with TTA, validation n=13), liver via nnU-Net v1 (5-fold ensemble, n=10 held-out)
Brain anatomy reference	Fixed dummy brain prototype; tissue extracted via Otsu thresholding and morphological operations	Patient-space segmentation plus Harvard–Oxford atlas-backed anatomic mapping with registration-quality awareness
Spatial alignment	Center-of-mass rigid translation of the tumor into the dummy brain	Center-of-mass rigid alignment retained, extended with atlas-space localization and quality flags
Quantitative tumor metrics	Not computed (segmentation mask only)	Total burden, max diameter, per-label volumes, lesion list, merged lesion-group count, segmented component count
Clinical reporting	Not implemented	Deterministic structured report with fixed sections (Clinical Context, Findings, Impression, Limitations, Recommendations, Disclaimer) and evidence references
AI assistant / QA	Not implemented	Local llama.cpp runtime with Qwen2.5-7B-Instruct (GGUF); evidence-linked QA with guardrails on diagnosis, treatment and prognosis questions
Evaluation framework	Not implemented	Automated per-case artefacts: quality_report.json, metrics.json, eval_summary.json, error_log.json, comparison.json, run_meta.json
VR visualization	Manual Unity scene setup with UnityVolumeRendering plugin deployed on Meta Quest 3	Automated Unity VR deployment pipeline with performance optimizations for Quest 3 (Fixed Foveated Rendering, 0.65× mobile sampling-rate multiplier (~333 effective steps) with tricubic interpolation disabled on Android, locked GPU clock, stable 72 Hz)
Desktop application	Not implemented	MedVisVR desktop application integrating the case viewer, report draft and AI assistant in a single interface
Network dependency	Segmentation runs locally, but pipeline not explicitly designed for disconnected use	End-to-end fully offline by design: segmentation, metrics, atlas mapping, reporting, LLM inference and VR all execute on local hardware with no cloud or external API calls
Reproducibility	Ad-hoc scripts without run-level fingerprinting	Per-case run_meta.json with code fingerprint, prompt version, LLM policy, question-set hash and artefact paths

2.3. Interpretation of Results

Brain Tumor Segmentation Performance

The nnU-Net v2 inference on the BraTS 2020 held-out validation split (n = 5) yields per-sub-region Dice in the 0.80–0.87 range with macro-F1 0.873 ± 0.057, and the 5-fold checkpoint EMA Dice of 0.8691 ± 0.0019 is internally consistent with the validation slice. The enhancing-tumor sub-region (label 4) is the most stable (Dice std 0.023), whereas NCR and edema show larger variance, mostly because of lower recall on a small number of weaker cases. These numbers fall within the regime of reported BraTS 2020 nnU-Net single-fold baselines (approximately 0.85–0.90) but below top-leaderboard ensembles with heavy post-processing. That gap is expected and intentional: segmentation here is a pretrained backbone that feeds the downstream interpretation and VR layers, not a benchmark submission.

Lung Nodule Segmentation Performance

The held-out validation mean Dice of 0.652 ± 0.271 on n = 13 cases is driven by bimodality, not by uniform degradation. The majority of cases cluster above Dice 0.78 (median 0.791) while one failure case (lung_033) registers Dice 0.0, a 115-voxel prediction against a 39,103-voxel ground truth (correct localization, but under-segmentation by two orders of magnitude). Reporting only the cohort mean would hide this split and suggest uniform accuracy; the per-case view makes the failure mode visible. Specificity saturates at 0.99994 because of the extreme foreground/background imbalance intrinsic to lung-nodule segmentation, which is why Dice, precision and recall (and not pixel-level accuracy) are the informative metrics here. In practice, any deployment of this system would need automated per-case quality checks rather than relying on a cohort-level average.

Liver Tumor Segmentation Performance

The 5-fold ensemble on the 10-case held-out liver cohort produces parenchyma Dice 0.955 ± 0.053 and tumor Dice 0.706 ± 0.357 (all 10 cases, lower-bound: two tumor-free cases assigned Dice = 0.0). Excluding liver_105 and liver_106, the tumor Dice on the 8 tumor-present cases is higher than the all-10 average, but the all-10 figure is reported as the conservative lower-bound. The large standard deviation (0.357) is driven by these two tumor-free cases and by one small-tumor case. The parenchyma number is in the expected nnU-Net range for a large, high-contrast abdominal organ. As an independent sanity check, nnU-Net's own 5-fold cross-validation on the training data (n = 131) gives parenchyma 0.9571 and tumor 0.6372; the held-out ensemble result (0.706) is approximately 0.07 higher than the CV estimate, which is within one pooled standard deviation and is consistent with the small held-out sample size rather than systematic overfitting. The main remaining gap is a larger, expert-annotated tumor cohort that would support a tumor-specific baseline rather than a mixed tumor/non-tumor average.

VR Visualization and Integration

The VR stack is interpreted not as a rendering engine in isolation but as a perceptual channel through which the segmentation results are received. Two decisions shape this channel. First, a 4× intensity enhancement on the enhancing-tumor sub-region combined with multi-scale Gaussian boundary blending makes the tumor boundary visible in the rendering without implying a sharp anatomical edge, which fits the diffuse infiltration pattern typical of glioblastoma. Second, pairing the volumetric view with a live 2D cross-section and an angle readout gives the

user the planar view clinicians routinely read, without leaving the headset. The Quest 3 performance envelope (72 Hz sustained via a 0.65× sampling-rate multiplier, disabled tricubic filtering, Fixed Foveated Rendering and locked GPU clocks) comes with a known trade-off: mild voxel aliasing in low-gradient regions and some risk of under-sampling sub-voxel structures. This is acceptable for educational use, but it should be taken into account when interpreting subtle boundary transitions.

AI Assistant and Clinical Decision Support

We evaluated the assistant on two dimensions: hallucination control and pipeline integrity. Hallucination control is enforced at three layers. A generation-time guardrails module blocks disallowed diagnosis terms and cross-checks every numerical claim against the PatientContext with a 2 % tolerance. A deterministic report generator rejects drafts that miss required sections or match forbidden treatment-prescriptive patterns. An automated harness classifies each answer on a reference question set as supported, hallucination or omission, and emits aggregate hallucination_rate, omission_rate and safety_compliance values per run. Case-locking at the session layer (history discarded on patient_id/study_id change or 1800 s idle TTL) rules out cross-patient contamination through conversational state. Pipeline integrity is measured by a pytest suite covering schema validation, report generation, QA engine, retrieval, case repository, API endpoints, integration and a golden-case sanity pass. The pipeline behavior tests pass across the board; the only open items are a small set of schema-validation tests affected by a known contract-drift between current tumor_metrics.json field names and legacy required keys. This is a validator-side issue scheduled for the next revision and does not reflect a pipeline regression.

2.4. Analysis of Limitations

Segmentation-Dependent Pipeline

A central limitation of MedVisVR is that everything downstream depends on segmentation quality. Quantitative metrics, atlas localization, report generation, QA and VR rendering all inherit the errors of the segmentation mask. If the model misidentifies tumor boundaries, under-segments edema or produces fragmented masks, the rest of the pipeline can remain internally consistent while still being clinically misleading.

This is especially problematic because segmentation errors are often subtle and may not be obvious in the VR environment. A clinician or student viewing a volumetrically rendered tumor has no built-in mechanism to assess whether the rendered boundaries accurately reflect the true pathology. Future work should add segmentation uncertainty visualization, for example using transparency gradients or color-coded confidence maps overlaid on the rendered volume.

Rigid Registration Limitations

Center-of-mass rigid alignment preserves tumor morphology but sacrifices anatomical precision. This trade-off is acceptable for educational use but limits the system for surgical planning. Deformable registration (ANTs, SimpleElastix) would improve anatomical fidelity, at a significantly higher compute cost and with some risk of distorting tumor shape. The visualization uses a standard brain prototype rather than the patient’s own anatomy, so spatial relationships between the tumor and patient-specific landmarks (eloquent cortex, motor areas, vessels) are not preserved. This is a significant limitation for any neurosurgical-planning use.

Atlas-Based Localization Uncertainty

Harvard–Oxford atlas mapping provides approximate neuroanatomical localization that degrades when registration quality is poor. Uncertainty-aware language softens the wording when support is weak, but localization descriptors should be read as orientation aids rather than definitive anatomical assignments, particularly near atlas boundaries or in regions with high inter-subject variability.

Dataset and Generalizability Constraints

While the project uses publicly available datasets (BraTS 2020 for brain tumor, and the Medical Segmentation Decathlon lung CT and liver CT datasets), several generalizability concerns remain:

- **Institutional bias:** The BraTS 2020 dataset, although multi-institutional, primarily represents glioma cases from academic medical centers. Performance on data from community hospitals or low-resource settings with different scanner protocols remains untested.
- **Pathology scope:** The system is currently validated for glioma (brain, BraTS 2020, n = 5 fold-0 cases), lung nodule (n = 13) and liver tumor segmentation (n = 10). Extension to other pathologies (meningiomas, metastases, vascular malformations) would require additional training data and validation.
- **Population diversity:** The demographic composition of the training datasets may not fully represent global patient populations, potentially introducing bias in segmentation performance across different ethnic groups or age ranges.
- **Temporal limitations:** The system processes single-timepoint studies. Longitudinal tracking of tumor progression or treatment response would require additional pipeline components.

VR Hardware and Accessibility

The Meta Quest 3, while offering excellent standalone VR capabilities, introduces several practical limitations. The mobile GPU imposes constraints on rendering quality and volume resolution. The 0.65× sampling-rate multiplier applied on Android reduces the effective ray-marching step count from 512 to roughly 333 and can under-sample structures thinner than the voxel step, which is a concern for millimetric lesions and thin septa. In addition, tricubic interpolation is disabled on Android for an approximately 8× reduction in texture reads per sample; the resulting voxel aliasing is most visible in low-gradient regions such as normal parenchymal transitions. Together these two mobile-only shader choices may cause subtle visual artifacts in regions of gradual density change. Additionally, the VR interaction paradigm, while intuitive for many users, presents accessibility barriers for individuals with vestibular disorders, visual impairments, or limited hand mobility.

The need for a PC workstation for preprocessing limits deployment in resource-constrained clinical environments; a cloud-based alternative would lift that constraint but reintroduce privacy and latency concerns.

Beyond GPU and ergonomic considerations, the Unity-side deployment itself introduces additional, more operational limitations:

- **Sideload-based case transfer:** Each new case must be copied into the headset's persistent data directory via USB sideload (adb push or Meta Quest Developer Hub). The system has no headset-side case browser or PACS/DICOM ingestion layer.
- **Single-scene, single-case architecture:** The VR build ships with a single Unity scene initialized for one volume at a time; switching cases effectively requires leaving and re-entering the application, precluding VR-side playlists and multi-case comparison within one session.
- **No in-VR annotation or measurement:** The interaction model supports sphere-driven slicing plane control and a transfer-function toggle, but no volumetric measurement, bookmarking or on-volume annotation.
- **Narrow transfer-function library:** Only two transfer-function presets are implemented (plugin default and CT-Lung); additional modality- or tissue-specific presets would require code changes.
- **No axial/coronal/sagittal snap on slicing plane:** The slicing plane is driven continuously by the user sphere; there is no preset snap to the standard radiological planes.
- **Controller-only interaction:** All user input is bound to Meta Quest controllers; hand-tracking is not used.

Evaluation Methodology Gaps

The current evaluation relies primarily on voxel-level segmentation metrics (Dice, Precision, Recall) and system-level quality checks. Several important areas are not yet covered:

- **Clinical utility assessment:** No formal user studies with clinicians or medical students have been conducted to measure the system's impact on spatial comprehension, diagnostic accuracy, or surgical planning outcomes.
- **VR usability evaluation:** System Usability Scale (SUS) scores and task completion time measurements have not been collected.
- **Learning outcome measurement:** Pre-test/post-test studies comparing VR-based learning with traditional methods have not been performed.
- **Long-term retention:** The effects of VR-based medical education on long-term knowledge retention remain unmeasured.

2.5. Ethical Considerations

Patient Data Privacy and Anonymization

MedVisVR uses only publicly available, de-identified, IRB-approved research datasets (BraTS 2020 and the Medical Segmentation Decathlon lung CT and liver CT datasets), which removes the primary-collection consent problem. Three residual privacy points remain relevant in practice:

- **Offline by construction:** Segmentation, metrics, atlas mapping, the FastAPI service, the local llama.cpp LLM and the VR all run on the workstation and headset without network dependency. No patient image, report or question ever leaves the local environment, which protects PHI by construction and allows deployment in low-connectivity settings.
- **Headset-side residuals:** NIFTI files written to the headset's persistent data directory remain until manually removed. There is no session-end purge, encrypted-at-rest storage or per-case access control at the VR layer. Shared headsets therefore need an explicit deletion / session-wipe protocol. Additionally, the Quest 3 platform layer (Meta account, OS-level updates, device telemetry) sits outside the application sandbox and must be operated in Meta Quest for Business / MDM mode for strict HIPAA/GDPR environments.
- **Visualization identifiability:** Skull-stripping mitigates re-identification via facial reconstruction, but any future extension to non-stripped data must revisit this. Immersive 3D renderings also carry higher recognition potential than 2D slices and should be anonymized appropriately for educational demonstrations.

Clinical Responsibility and Liability

MedVisVR is explicitly positioned as a research and educational decision-support prototype rather than a clinical diagnostic system. This distinction matters for several reasons:

- **Regulatory status:** The system has not undergone the regulatory approval process required for clinical medical devices (CE marking in Europe, FDA clearance in the United States). Using it for real clinical decisions without that approval would be both ethically and legally problematic.
- **Disclaimer framework:** The report generation pipeline includes mandatory disclaimer sections and safety statements, which is a responsible approach. However, the effectiveness of these disclaimers depends on users actually reading and understanding them, which cannot be guaranteed in practice.
- **Over-reliance risk:** The professional quality of the VR visualizations and AI-generated reports could create a false sense of clinical reliability. Users may be tempted to treat the system's outputs as validated clinical findings rather than research-grade approximations.

Algorithmic Bias and Fairness

Deep learning models trained on specific datasets inevitably reflect the characteristics and biases of those datasets. For MedVisVR, the ethical implications include:

- **Demographic representation:** If the training datasets disproportionately represent certain demographics, the model's segmentation accuracy may vary across patient populations. This could lead to disparate quality of visualization and decision support for underrepresented groups.
- **Pathology spectrum:** The system's focus on glioma, lung nodule, and liver tumor segmentation means that other brain pathologies may be inadequately represented or misinterpreted if the system is applied beyond its validated scope.

- **Transparency:** The system's evidence-linked outputs and confidence scoring provide a degree of algorithmic transparency that helps users assess the reliability of individual outputs. However, current transparency is limited to aggregate confidence scores; voxel-level explainability and audit trails remain absent and should be addressed in future work to meet clinical accountability standards.

Informed Use in Medical Education

When deploying MedVisVR in educational settings, ethical considerations include ensuring that students understand the system's limitations and do not develop uncritical trust in AI-generated medical outputs. The system should be presented as a tool that augments, rather than replaces, traditional medical education methods. Educators should emphasize that the VR visualizations are approximations derived from automated segmentation and should always be validated against clinical expertise.

Intellectual Property and Licensing Compliance

The project adheres to the specific licensing terms of each dataset (primarily Creative Commons Attribution 4.0 International). The use of open-source tools (nnU-Net, UnityVolumeRendering, llama.cpp) also requires compliance with their respective licenses. The scientific integrity commitment to referencing all original authors and institutions ensures proper attribution and reproducibility.

VR-Induced Discomfort and Informed Consent

The VR layer introduces an ethical consideration that has no direct analog on the desktop side: head-mounted display use can produce cybersickness, eye strain and disorientation, with a non-negligible incidence in first-time users and in individuals with pre-existing vestibular or ocular conditions. The system’s performance optimizations (stable 72 Hz refresh rate, Fixed Foveated Rendering, mobile-tuned sampling rate) are designed to reduce this risk at the engineering level, but they do not remove it. In educational deployments, particularly where medical students, trainees or patient participants are asked to use the headset as part of a teaching session or user study, this implies a specific informed-consent obligation: participants should be told in advance that the session involves immersive VR, what the known side effects are, that they may stop at any time without consequence, and that time-limited exposure and rest breaks are available. This consideration should be reflected in any institutional review protocol accompanying the deployment of MedVisVR in a learning context.

2.6. Scalability Analysis

Computational Scalability

The MedVisVR pipeline has several computationally heavy stages. Scaling to larger deployments requires addressing the following:

Segmentation Throughput

nnU-Net inference on a single brain MRI case with GPU acceleration takes approximately 1-2 minutes, while CPU inference can exceed 30 minutes. For clinical deployment scenarios requiring rapid turnaround, this inference time may be acceptable with GPU acceleration but is prohibitive on CPU-only systems. Scaling to high-throughput environments would require either dedicated GPU infrastructure or cloud-based batch processing.

VR Rendering Performance

The Meta Quest 3's mobile GPU (Adreno 740, part of the Snapdragon XR2 Gen 2 SoC) constrains the rendering complexity. The current optimization strategy (a 0.65× sampling-rate multiplier, corresponding to roughly 333 effective ray-marching steps down from 512; tricubic interpolation disabled on Android; Fixed Foveated Rendering; 72 Hz refresh rate) achieves acceptable performance for single-volume visualization. However, simultaneous rendering of multiple volumes, higher-resolution datasets, or additional anatomical overlays would strain the available GPU budget and may require more aggressive level-of-detail strategies or streaming approaches. Three further scalability boundaries follow directly from the present Unity implementation. First, the Quest 3 exposes roughly 8 GB of shared memory with a practical per-application ceiling below that, so a single float-16 volume at 512^3 already occupies on the order of 268 MB before counting slice render targets, shader-side atlases and UI; 1024^3 volumes or multi-modal overlays (MRI + CT fusion) would exceed the budget without tiling or downsampling. Second, the runtime volume bootstrap is a single-shot builder: it tears the previous sphere down and constructs a new GameObject hierarchy on load, which is adequate for case-at-a-time review but is not a hot-swap or streaming architecture. Third, loading large NIFTI files from the headset's internal storage introduces a user-visible latency that can reach tens of seconds for volumes around 500 MB; the on-device loading HUD mitigates the user-experience impact but does not remove the underlying I/O bottleneck. Closely related is PC-tethered rendering via Quest Link / Air Link, which would lift most of these mobile limits at the cost of reintroducing a host-to-headset dependency.

Multi-Pathology Extension

Expanding MedVisVR beyond brain glioma, lung nodule, and liver tumor segmentation to support additional pathologies is architecturally feasible but requires:

- **Additional training data:** Each new pathology type requires curated, annotated datasets and dedicated model training or fine-tuning.
- **Transfer function design:** New pathology types may require custom transfer functions for optimal VR visualization.
- **AI assistant adaptation:** The structured descriptor layer, question classification, and safety rules would need to be extended for each new clinical domain.
- **Validation studies:** Each pathology extension would require independent validation to ensure segmentation accuracy and visualization quality meet established standards.

Institutional Deployment

Deploying MedVisVR in clinical or educational institutions presents several scalability challenges:

- **IT infrastructure:** Institutions would need CUDA-capable workstations for preprocessing, Meta Quest 3 headsets for visualization, and potentially network infrastructure for data transfer.
- **Data integration:** Clinical deployment would require DICOM-to-NIfTI conversion pipelines and integration with hospital PACS systems.

- **User training:** Both the desktop application and VR interface require user training. More comprehensive onboarding workflows would be needed.
- **Maintenance and updates:** Keeping segmentation models current, updating VR applications, and maintaining hardware would require dedicated technical support.

2.7. Comparison with Existing Approaches

Within the existing literature, the contribution of MedVisVR is integrative rather than foundational. The system does not introduce new segmentation architectures or new VR rendering techniques. Its value lies in the end-to-end combination of established methods into a coherent, usable pipeline.

Compared with structured clinical reporting frameworks such as BT-RADS, MedVisVR is less clinically mature but more automated at the case-processing level. Compared with LLM-first report generation approaches, the system is more conservative and more tightly grounded in deterministic case data. Compared with segmentation-only studies, MedVisVR contributes less at the model-innovation level but significantly more at the level of explainability, downstream usability, and immersive visualization.

The main differentiator is the integrated, fully offline pipeline: automated segmentation, quantitative analysis, atlas-aware localization, grounded reporting and interactive VR visualization, all executed locally without cloud dependencies. Many comparable systems rely on cloud-hosted LLM APIs or remote inference services, which creates patient-privacy friction and makes them unusable in disconnected clinical environments. We are not aware of a publicly documented system that combines these capabilities in a single end-to-end, locally executable workflow covering both standalone VR visualization and desktop-based clinical review.

2.8. Future Directions

Based on the limitations and scalability analysis presented above, the following directions are identified for future development:

- **Segmentation uncertainty visualization:** Incorporating voxel-level confidence maps into the VR rendering to provide users with intuitive understanding of segmentation reliability.
- **Deformable registration:** Implementing more sophisticated spatial alignment methods to improve anatomical accuracy for surgical planning applications.
- **Clinical validation studies:** Conducting formal user studies with clinicians and medical students to measure the system's impact on spatial comprehension, learning outcomes, and planning accuracy.
- **Multi-pathology support:** Extending the pipeline to support additional neurological and non-neurological pathologies.
- **Longitudinal tracking:** Adding capabilities for temporal comparison of sequential imaging studies to support treatment response monitoring.

- **Multi-user collaboration:** Implementing networked VR sessions for collaborative surgical planning and group medical education.
- **Regulatory pathway:** Initiating the regulatory approval process to transition the system from a research prototype to a clinically deployable medical device.

2.9. Conclusion

This chapter examined MedVisVR across four areas: experimental results, limitations, ethical considerations and scalability. The system brings together deep-learning segmentation, quantitative analysis, atlas-aware localization, grounded clinical reporting and immersive VR visualization in a single pipeline.

The segmentation results, per-sub-region Dice of 0.80–0.87 for brain, 0.95 parenchyma and 0.71 tumor for liver, and a bimodal 0.65 mean Dice for lung driven by one under-segmentation failure, sit in the range expected from pretrained nnU-Net backbones and are sufficient for educational and research use. The VR stack, with its 4× intensity enhancement, multi-scale boundary blending and interactive slicing, provides intuitive access to complex 3D medical data on affordable standalone hardware.

Significant limitations remain: segmentation-dependent error propagation, rigid-registration accuracy, the absence of clinical validation, and Quest 3 hardware and accessibility constraints. The ethical analysis reinforces the research-prototype positioning, the privacy guarantees (and the platform-layer caveats around them), and the need to address algorithmic bias. The scalability analysis identifies both technical ceilings (GPU budget, storage I/O, single-scene architecture) and extension paths (multi-pathology, institutional deployment, PC-tethered rendering).

Overall, MedVisVR is a working prototype for accessible, immersive medical visualization. Taking it from a research prototype to a clinically validated tool will require further work on validation, regulatory compliance and user-centered design.

3. Conclusion & Future Work

3.1 Contributions

The CARVIS project provides a full-stack clinical AI platform that integrates deep learning-based medical image segmentation with an interactive, physician-facing application. The contributions cover two closely coupled layers, the segmentation pipeline and the application layer that shows segmentation results to end users.

Segmentation Pipeline

Multi-Organ, Multi-Framework nnUNet Integration

Three independent segmentation modules were designed, implemented and validated, each targeting a different anatomy of clinical interest and using a framework tuned to the data characteristics of that anatomy:

- Brain tumors: nnUNet v2, BraTS 2020 dataset, 5-fold ensemble over the folds 0-4, providing sub-region predictions of enhancing tumor (ET), non-enhancing core / necrosis (NCR/NET) and peritumoral edema (ED).
- Liver parenchyma and tumors: nnUNet v1 (Task003_Liver), 5-fold ensemble, Labels: background, liver, liver tumor.
- Lung tumors: nnUNet v2, Medical Segmentation Decathlon Task06, single-fold checkpoint.

Due to the incompatibility of the Python environments of nnUNet v1 and v2, a dual-environment architecture was adopted, where the liver module spawns an isolated subprocess under a dedicated virtual environment, while the brain and lung modules operate via the Python API within the main application environment.

BraTS Sub-Region Label Normalization

In the BraTS labeling scheme, values 1, 2, and 4 correspond to non-enhancing core/necrosis (NCR/NET), peritumoral edema (ED), and enhancing tumor (ET), respectively. Label 3 is intentionally unused in the BraTS convention, and the pipeline explicitly discards any voxels carrying this value to prevent contamination of downstream metrics. During metric computation, reporting, and uncertainty estimation, all numeric labels are mapped to their clinical names so that the sparse, non-contiguous label space cannot be silently misinterpreted as a dense integer range.

PCA-Based Longest-Axis Diameter Measurement

Rather than determining the diameter from the bounding-box edge (that tends to overestimate non-spherical lesions), the pipeline determines the length of the lesion's projection on the first principal component (PC1) of the point cloud in millimeters. Bounding-box calculation acts as a fallback procedure if the lesion consists of fewer than three voxels or if the covariance matrix is singular.

Harvard-Oxford Atlas Localization

Each lesion is anatomically labeled by aligning the patient's image to the MNI152 1mm template using the SimpleITK rigid and affine transformation, with histogram matching and the Mattes mutual information metric. The new centroid of the lesion in the aligned image is used for querying the Harvard-Oxford cortical-lateralized and subcortical atlases for assigning the lobe, hemisphere, and gyrus labels, along with a registration quality score; the normalized patient's voxel coordinates serve as the last-resort option if alignment or atlas lookup fails.

Dice-Validated Confidence Intervals

The volumetric confidence intervals are calculated based on the internally validated Dice scores rather than applying the uniform ±3% band; for each tissue type, the uncertainty coefficient is set as max(1-Dice, 0.03):

- Enhancing tumor: Dice 0.87, uncertainty ±13%
- Non-enhancing tumor: Dice 0.80, uncertainty ±20%
- Edema: Dice 0.82, uncertainty ±18%
- Lung tumor: Dice 0.65, uncertainty ±35%

- Liver parenchyma: Dice 0.95, uncertainty ±5%
- Liver tumor: Dice 0.60, uncertainty ±40%

Previously, a hardcoded ±3% band was applied to all label types, resulting in significant underestimation of model variance by an order of magnitude for highly variable labels like edema and lung lesions.

Non-ASCII Path Resilience

On Windows systems, where the localization is configured to Turkish, non-ASCII characters are included in the default name of folders (e.g., "Masaüstü" for the Desktop); this causes the NIfTI file writer to silently terminate without providing any feedback. To ensure operation irrespective of the OS localization or OneDrive configuration, a tiered approach was designed: (i) if the path is ASCII-safe, keep it; (ii) otherwise, derive the legacy short path from the system to retrieve the ASCII path; (iii) finally, write the file into the temporary directory and then move it to its destination path.

3.2 Application Layer

FastAPI REST Backend

The backend serves the following structured REST API interfaces, via six domain-scoped routers: cases, CT cases, viewer, report, Q&A, and upload. All API endpoints serialize/deserialize and validate request and response bodies using Pydantic schemas, and each API is automatically documented with OpenAPI via FastAPI, easing external integration efforts in the future.

Intent-Driven Clinical Q&A Engine

A multi-layer natural language question and answering engine takes free text clinical questions about a loaded case. The pipeline applies (1) a spell correction layer using a domain-enriched medical vocabulary; (2) a local intent classification against a typed intent tree with intents for quantitative, spatial, comparative, prognostic and management domains; (3) optional LLM-based intent arbitration for ambiguous questions, and (4) session memory to carry resolved intents across turns in order to facilitate multi-turn conversations.

Automated Structured Report Generation

The report generator is clinician-facing structured reports with six required sections: Clinical Context, Findings, Impression, Limitations, Suggested Clinical Correlation, and Disclaimer; and nine Findings sub-sections: volumetric analysis, spatial localization, confidence scores, VR-ready status, mass effect, midline shift, pattern of enhancement, and multifocality. Every report includes an LLM-generated narrative when it exists, with rule-based alternatives if it does not.

Clinical Safety Guardrails

A specific guardrails module implements three safety constraints on all generated outputs: (1) recognition and blocking of definitive diagnostic terms (e.g., "This is definitely glioblastoma"), (2) generation of low-confidence outputs triggers a required, low-evidence safety note, (3) all clinical outputs fall under one or more rule sets for blacklisted semantic structures before being shipped to the client.

Electron Desktop Application & One-Click Deployment

A desktop application based on Electron encapsulates FastAPI API and React user interface to provide an entirely offline, physician-facing Graphical user interface. Short, automated scripts ensure the virtual environment exists, all dependencies are installed, optionally fire up an LLM server, automate a health-check poll, and make the initial first-time install on a new clinical PC a single double click and no configuration. Uses a separate, first-run setup script to automate the full dependency install process for a non-Python environment.

Immersive VR Volume-Rendering Module (Meta Quest 3)

In addition to the Electron desktop client described above, a standalone Meta Quest 3 application (MedDemo) is shipped that takes the same NIfTI outputs from the desktop pipeline and renders them as fully interactable 3D volumes inside a head-mounted display. Built in Unity 6 (6000.3.7f1) on the Universal Render Pipeline, this application includes the Meta XR SDK 85.0.0 to provide native hand tracking and controller input, and uses a heavily forked open source UnityVolumeRendering package as its raycasting implementation. The build is exclusively Quest 3 (ARM64, multiview stereo, 90 Hz) and is sideloaded as a single signed APK.

The VR module makes the following concrete contributions to the wider CARVIS platform:

- **Runtime NIfTI ingestion without rebuild:** A new bootstrap component (RuntimeVolumeBootstrap) will decompress patient volumes at run time using a 3-step path resolution (explicit absolute path-> Quest persistent data path->bundled StreamingAssets). New cases can be deployed to the headset by adb-pushing a .nii or .nii.gz file into the application persistent data folder; no apk rebuild, app reinstallation or dev machine is needed in between patients.
- **Auto-fitted, hand-graspable volumes:** Whether a volume is grab-able, or "hand-graspable", is determined dynamically during compression. Its size is normalized to the package's bounds-fit primitive, then remains thus rescaled so the world-space length of the longest edge is about 0.35 m an empirically determined size to fit the natural arm span of the user. Two independent grab affordances (VolumeGrab, a grabable grid-aligned volume container with radius 0.28 m, and a sphere "handle" for the cross section tool with radius 0.12 m) are attached.1 A static co-ordinator prevents both grab-able features from being caught at the same time redundant input in the most common nested grab hierarchy cases.
- **Dual-input interaction model:** VolumeGrab can accept Touch controllers (grip / trigger) and bare-hand pinch gestures via OVRHand. The same scene can be used by clinicians in either input mode without recompilation (since the hand-tracking path is superimposed on the controller path). This is particularly useful for sterile or peri-op review.
- **Object-driven cross-sectioning:** A spherical handle is the combined location of a CrossSectionPlane (clipping the volume) and a SlicingPlane (rendering the cut as a 2D MPR-style image on a world-space quad). A single rig controller (RuntimePlaneRigController) drives the two planes so that the user only has to interact with a single"object, rather than two coupled gizmos. To prevent the volume from blinking out of existence as the application loads, the cross-section is gated by a

movement threshold (ActivateCrossSectionOnSphereMove): volume clipping doesn‘t activate until the sphere has moved a tiny distance, ensuring that the user always starts out seeing the full volume.

- **Clinical transfer-function presets with controller toggle:** Three 3D Slicer-derived presets are provided -- CT-Lung, CT-Cardiac3, and CT-Chest-Contrast-Enhanced, encompassing most common clinical scenarios of thoracic CT; specifically, chest, cardiac, and contrast-enhanced thorax CT. Clicking the Quest right-controller B button toggles between each preset in the order of Default -> CT-Lung -> CT-Cardiac3 -> CT-Chest-CE -> Default. The toggle uses the New Input System action-binding API with a polling-based XR fallback that keeps the toggle working on devices where action-map activation is deferred.
- **Quest 3 GPU and shader-stripping countermeasures:** A mobile-target shader optimizer (Quest3ShaderOptimizer) reduces the volume sampling rate to 0.65 and disables tricubic filtering on Adreno-class GPUs, maintaining the point of rendering at a higher refresh rate reprojection while still remaining inside the headset thermal envelope. Three runtime-instantiated shaders (the URP volume shader, the slicing-plane shader, and the custom 2D HUD shader) are explicitly registered in the project‘s Always-Included Shaders list to ensure the build pipeline does not strip them from the build when none of the scene assets reference them at compile time.

3.3 Lessons Learned

CARVIS development revealed many engineering and scientific lessons regarding clinical AI systems that are universally applicable to them.

Segmentation & Model Integration

- **Lesson 1: Compatibility issues with Framework version is a first class deployment concern.**

Running nnUNet v1 and nnUNet v2 side-by-side in the same python environment results in quiet import errors and incorrect results. Running one framework per virtual environment, and having them talk through subprocess creates a sane, extendable solution.

- **Lesson 2: Fixed estimates of uncertainty can be deceptive to the clinician.**

Initial design employed a fixed +/-3% confidence band across all tissue labels. Internal calibration showed this under-represented the ‘true’ segmentation uncertainty by anywhere up to 6x for high-variance tissues (brain edema: Dice 0.82 -> true error 18%). Anchoring of the confidence band to statistically validated Dice scores vastly increased report credence.

- **Lesson 3: The avgs do not tell you what you need to know. Per-case variation needs to come to the fore, not be smoothed away through averaging.**

Lung Tumor Segmentation achieved an average Dice of 0.65, a standard deviation of 0.27, and a median of 0.79. These statistics showed a highly bimodal distribution. Showing only the average flattenes the distribution because the system performs terribly on some cases. Confidence (High/Moderate/Low) labels were added to inform end users of this variability.

- **Lesson 4: Small tumors of liver: a still open issue**

Liver tumor segmentation (Dice 0.60) was consistently the lowest in performance. Many sub-centimeter lesions were not detected which is a common limitation of present day nnUNet training data rather than an inference pipeline. It emphasizes the importance of having to make people aware of model limitations in generated reports.

System Engineering & Deployment

- **Lesson 5: Filesystem encoding is an invisible killer in production.**

OneDrive paths on Turkish-locale Windows systems have non-ASCII characters (“Masaüstü”, “Kullanıcılar”) that cause SimpleITK NIfTI writer to crash without producing a comprehensible error message. Identifying and avoiding these paths before a data-loss failure can occur is valuable in pragmatic deployment environments.

- **Lesson 6: A offline first architecture is almost a requirement for a clinical environment.**

Hospital environments often control outbound access to the internet. Making the LLM feature a fully optional upgrade (with robust rule based fallbacks) rather than an integral part guaranteed that it would work in many different deployment scenarios.

- **Lesson 7: domain specific NLP preprocessing cannot be done by general libraries.**

It took a significant effort to create a qualifying medical vocabulary extension as otherwise correctly spelled words, such as ;, enhancing, and glioblastoma are marked as errors by standard spell-checks and it is impossible to properly classify the intent of a medical question.

- **Lesson 8: LLM guardrails must be explicitly forbidden-pattern enforced, not just prompted.**

Giving the LLM a system prompt to “avoid definitive diagnoses” mitigates but doesn‘t completely prevent unsafe outputs. The additional post-generation validation layer to compare against a predefined collection of “forbidden patterns” gives a deterministic safety net independent of model behavior.

- **Lesson 9: Mobile XR silently builds strip shader instantiated by running them.**

Volumerendering assets in CARVIS are generated on-the-fly by the application, they are not authored within a scene. Consequently any build pipeline for Unity treats the relative shaders as unreferenced, removing them from the APK. This results in a functioning editor build however it does not render the volume on the headset. A reliable way to overcome this is to add all runtime shader GUIDs in ProjectSettings/GraphicsSettings.asset into m_AlwaysIncludedShaders, although URP shader GUIDs are known to vary and this will need to be re-verified after each package update.

- **Lesson 10: Quest 3 GPU budget is very tight for direct volume rendering.**

A naive port of the desktop volume shader to Quest 3 achieved single digit frame rates and caused Asynchronous SpaceWarp, both of which are unacceptable in a clinical scenario. Three mitigations altogether brought the target refresh rate back to a stable higher value reducing the maximum number of steps, not reducing the mobile sampling rate to 0.65, and disabling tricubic

interpolation on Adreno GPUs. URP render scale was also lowered to 0.8 with 4x MSAA, giving a perceptually better trade off than rendering at full resolution without MSAA.

- **Lesson 11: Sideload-friendly data layouts collapse the patient-rollout cost.**

A traditional pipeline would necessitate rebuilding and resigning an APK (using a call such as "android.bat -xf android-debug.apk AndroidDebug.apk") for each additional patient case. First resolving NIfTI paths dynamically to the Quest's persistent data directory eliminates this deployment loop, resulting in a single "adb push case.nii.gz", and restart of the application. This is a (relatively) inexpensive engineering change with outsized clinical influence turning the headset from a static-content demo unit into a versatile, per-patient diagnostic that can be refreshed on the order of seconds by any medical professional.

3.4 Applications

CARVIS is envisioned as an inclusive clinical AI infrastructure platform. The fundamental ability to abstract highly complex goals into practicable units underpins very tangible application cases for clinical practice, medical education, research, and public outreach. We have developed a showcase website (produced using Next.js14, React Three Fiber, and interactive 3-D organ atlas pages) that introduces the platform not only to users of existing installations but also to clinicians, hospital administrators, and researchers seeking an understanding of the platform before installation.

3.4.1 Clinical Practice

Radiology Decision Support

Automated calculations of tumor volume, maximum tumor diameters based on PCA measurement, tissue sub-region breakdowns, anatomic location are returned to the radiologists within seconds after cases are uploaded for review on MRI studies. The structured report format with a mandatory limitation/disclaimer section ensures added AI findings are a supporting evidence, not an end diagnosis.

Neurosurgical Planning

The Harvard-Oxford atlas localization module determines if a tumor borders or invades the functionally eloquent cortex (e.g. frontal motor areas, temporal language areas). Location relative to eloquent cortex, estimate of midline shift, and evaluation of mass effect all together can show a pre-operative overview that can be compared to basic imaging in the patient.

Oncology Longitudinal Monitoring

Standardized volumetric metrics with Dice-validated confidence intervals provide inter-visit comparability. The batch processing engine allows serial follow-up scans to be processed as a group, producing individual case-level JSON metrics files that can be imported into a hospital information system or research database.

Resource-Limited and Offline Settings

As the entire system of CARVIS is based on local hardware without any reliance on any cloud infrastructure, it can be implemented in developing countries in district hospitals, mobile imaging

vans or even on low bandwidth facilities. Utilization of GPU accelerations (with fallback to CPU when a GPU is not available) also allows the system to operate on regular clinical workstations.

Immersive Pre-Operative Planning in VR

Surgeons also use the Meta Quest 3 client to explore a patient CT or MRI volume in full stereoscopic 3D, move it freely at one-to-one hand scale, and reveal any internal anatomy with a hand manipulated cross-section. When contrasted with review on traditional flat-panel multi-planar reformats, this headset preserves depth perception and supports full bimanual interaction a key advantage in spatially demanding cases such as thoracic and cardiothoracic or craniofacial surgery. Because the volume is loaded from the headset‘s local storage and the app is independent of network connectivity, the workflow is readily deployable directly in the operating room anteroom or bedside both of which regularly lack reliable cloud network connectivity or an available clinical workstation.

3.4.2 Medical Education

Interactive Case-Based Learning

The Q&A engine repurposes all the stored cases into interactive didactic cases, allowing medical students and radiology residents to learn by asking questions about tumor volume, anatomic site, Enhancement patterns and differential considerations in natural language and receiving rich, well-organized evidence-grounded responses. Memory for sessions facilitates multi-turn pedagogical discussions that shape clinical logic stepwise.

Segmentation Quality Assessment Training

Confidence intervals and model validation metadata are directly accessible to the user, imparting knowledge about the limitations of the automation. It also prepares for future clinicians to question automation, rather than blindly accept it, a skill increasingly expected by medical AI regulation.

VR-Based Anatomy and Volume-Rendering Practice

The same Meta Quest 3 app that allows clinical pre-operative planning is also a nearly-free volumetric anatomy lab for medical students and radiology residents, allowing them to load and freely translate and scale any educational CT or MR dataset with their hands, toggle between all three clinically-relevant transfer functions (lung, cardiac, contrast-enhanced soft tissue) at a single button press, and use the cross-section tool to interactively demonstrate the connection between surface anatomy and internal structures. Since each transfer function preset is associated with a standard 3D Slicer clinical convention, the student is given an intuition for the same windowing used in regular practice, though it is reinforced by the headset‘s stereoscopic depth cues that make volumetric relationships easier to understand than on a flat monitor.

3.4.3 Research

Batch Cohort Analysis

A batch evaluation harness allows large case cohorts to be fed through a stat generator program that creates standardized metric files for each case that can be used directly for subsequent statistical analysis, model comparison, and cleaning of the dataset.

Model Development Scaffold

The modular segmentation architecture also means that new nnUNet checkpoints can easily be added to CARVIS by adding one Python module and registering a new API path. CARVIS becomes a rapid prototyping environment for clinical AI researchers who want to quickly develop and validate new segmentation models.

3.4.4 Outreach and Knowledge Translation

CARVIS Showcase Website

The showcase site was created along with the clinical desktop application in order to show other clinicians, hospital administrators, and researchers its functionality to those who may not have the installed desktop tool available to them at their institution. While the desktop version was created in our standard environment of Electron, React and D3, the showcase was created in Next.js14 (App Router, Typescript) using Tailwind CSS. Motion animation was handled in an all-in-one Anime.js library rather than in individual React components via Framer Motion so that all timing and easing remained the same on every page. Modality-specific 3D organ atlas pages (brain, liver, and lung) utilized React Three Fiber allowing users to hover and click on a 3D model to examine a specific organizer within the context of an anatomical region and read treatment-oriented clinical descriptions. The first view of the homepage featured a particle brain, created in Spline, which moved in response to the mouse cursor to give a taste of the neuroimaging focus of the platform at a glance without requiring any special technical knowledge.

There is also a separate page for platform architecture (the technology stack, pipeline summary, system diagram), segmentation execution performance (per-organ Dice results with confidence intervals and all), and a contact page with a feature for contacting the developers through the Resend email API. The pages listed above use content separated into per-organ TypeScript modules in a content/ directory, so that if a new organ for segmentation is added (e.g., prostate or kidney), only a new content module, and a new path registration are necessary; the render layer does not have to change. This separation of concerns in the website architecture reflects the modular approach of the clinical pipeline as well, so that the pipeline and website evolve in parallel as new disciplines, organs, validations, and releases are achieved.

3.5 Next Steps

The priorities for development presented below have been concluded based on existing system shortcomings, user feedback examples and the overall clinical AI framework.

Segmentation Enhancements

- Longitudinal alignment and delta reporting: automatically co-register serial scans from the same patient and calculate signed volume change and growth-rate estimates between visits.
- Broader organ coverage: to include kidney, prostate, pancreas and colon, add segmentation modules for them with the existing MSD and nnUNet benchmark datasets.
- Liver tumor model enhancement: the proprietary nnUNet test set Dice of 0.60 for liver tumor is inadequate for clinical applications requiring a high degree of certainty.

Additional fine-tuning on more cases or switching to a more current checkpoint of the nnUNet v2 is a short-term objective.

- Uncertainty visualization: show the per-voxel uncertainty as a heat map overlay in the 3D viewer so clinicians can find out spatially where model is most uncertain.
- More extensive validation: validate the model on a larger validation cohort than the n=5 (brain) and n=13 (lung) cases, providing a statistically reliable (Dice) estimate, preferably by tumor grade size, and MRI protocol.

Application Enhancements

- Native DICOM viewer integration the pipeline at present is NIfTI based. Native DICOM series viewer, can remove the conversion process for the hospital borne studies.
- PACS / HL7 FHIR communications: connection to hospital PACS systems through DICOM SR or HL7 FHIR diagnostic report resources would enable the CARVIS findings to be included in the electronic health record.
- Turkish-language report generation: the current Q&A engine supports Turkish by leveraging translingual intent match. Moving to fully Turkish-language structured reports would make reports easier for in country clinical users.
- Multi-user access and audit logging: before entering multi-physician setting or formal clinical trials, a role-based access control layer with non-modifiable audit logging is needed.
- Larger embedded LLM-the current deployment uses 7B-parameter GGUF models, avoiding demands on standard hardware of clinical workstations. Once GPU memory grows on commodity hardware, an upgrade to 13B-32B models will result in significantly better quality of report narratives and dialogues/Q&A.
- A responsive web interface would enable most bedside case review to be performed on tablets, which represents much of the workflow during ward rounds. This would also be equally usable from mobile phones.
- Multi-volume and segmentation overlay in VR: Currently, the Quest 3 client only displays one grayscale volume at a time. It would be nice if the output mask of the segmentation pipeline could be visualized as a color overlay (using per-label opacity controls) over the source volume(s), providing surgical teams an opportunity to view the structures the AI detected in 3D stereoscopically, pre-operatively.
- Bimanual volume scaling: the current Quest 3 volume client loads each volume scaled to a fixed largest-edge size; it supports translation/rotation via single hand grab. a bimanual pinch-to-scale gesture-a combined two-hands grab that scale the volume linearly based on the inter-hands distance-might allow surgeons to zoom in on small lesions for fine-grained inspection or pull back for an overview of full anatomy within the immersive view without reaching out and grabbing a menu.

Validation & Regulatory

- Prospective clinical study: a reader study will measure the time-to-report, measurement reproducibility, and diagnostic confidence of radiologists with and without CARVIS assistance.

- Ethics and data governance framework: for clinical deployment, an approved IRB data de-identification plan, de-identification process and model governance documentation in accordance with the applicable MDD regulations (EU MDR, FDA SaMD guidance).

3.6. Conclusion & Future Work

Summary

CARVIS is an end-to-end clinical AI system that extends the cutting edge of deep learning segmentation with a ready-for-deployment clinician-facing app. From an initial raw NIfTI upload all the way to receiving a fully structured radiology report with Dice-validated confidence intervals and atlas-based anatomic localization, the entire pipeline runs on an entirely offline workstation, accessible without the reliance on the cloud or high end DevOps skillsets. A related demo website (interactive 3D organ atlas pages, modality-specific performance tables, modular Next.js app architecture) invites clinicians and evaluators to dabble in the system before deploying it for themselves.

The system supports 3 anatomical targets: brain tumors (BraTS sub-regions), liver with tumors, and lung tumors with all 3 validation on public benchmark datasets; the application layer pairs these models with an intent-driven Q&A engine, a guardrail-protected report composition, and a one-click deployment stack that reduces the clinical adoption barrier. In tandem with this analytic/report pipeline, the Meta Quest 3 client wraps the same volumes for stereoscopic, hand-driven inspection on an offline headset, linking the platform from raw image to written report to immersive surgical pre-review.

Broader Impact

The long term benefits of the proposed CARVIS system transcend the speed advantages of a computer. The designed system, by consistently and reliably providing quantitative, reproducible measurements offers to help hospital staff regarding one of the widely persisting problems of medical imaging of patients- inter-observer variability. It has been established by research that radiologists' manual measurements of tumor volume exhibit large inter- and intra-observer variation and the proposed computerized segmentation with known quantified uncertainty establishes a more trustworthy reference point for treatment response.

For physicians (and prospective physicians) right now, we will reduce the burden of quantitative analysis. For instance, by providing radiology residents the ability to access this Q&A engine while working on a case, and receiving a concise, direct answer that is based directly off of the segmentation data from that case (rather than some standard textbook description), radiology trainees can query the Q&A engine while reviewing a case and receive answers derived from that case's segmentation output. Compared to static reference material, this case-grounded feedback shortens the loop between observation and explanation.

In Resource-Limited Settings, such as regional hospitals, mobile imaging centers, and hospitals in the developing world, the offline-first, 1-click deployment paradigm can make AI-supported diagnostics available to much broader populations that have historically only had access through

well-endowed academic medical centers. The lack of cloud subscription, internet connection, or fee-for-inference, removes traditional barriers to adoption at the point of care.

This Meta Quest 3 client, the companion to this piece, further extends this democratization argument from analytic AI into immersive space visualization. Volumetric preoperative review has been limited traditionally to dedicated (or cloud subscription operated) surgical-planning workstations and platforms costing tens of thousands of dollars a seat, limiting primary use to tertiary academic centers. A consumer grade standalone headset with a sideloadable APK drops that cost by roughly two orders of magnitude, removes the cloud dependence, and removes the specialized PC workstations, while the same device can serve as a stereoscopic anatomy laboratory for trainees, collapsing immersive 3D from the acquisition to a flagship-only feature to something that a regional hospital, a residency program in a low-income country, or a combat support hospital can use routinely.

AI as a Copilot, Not a Replacement

Guiding principles of the CARVIS design are that the system should never replace, but always assist the clinician. To that end, at several levels in the design, these principles are invoked. The guardrails module prevents forbidden diagnostic phrases, while all quantitative measurements are issued with a Dice-verified measure of uncertainty. Structured reports must include a 'Limitations' section and a warning about clinical correlations. Confidence labels (High / Moderate / Low) present a barrier-free, automatic presentation of segmentation confidence without necessitating the user understands Dice.

This policy isn't just a regulatory measure; it is a recognition-based understanding that almost all of today's deep learning approaches, including nnUNet, will sometimes give a failure mode unintuisvely to a non-expert end-user. Open communication of uncertainty is how this device is earned, not taken for granted.

Future Vision

In the short term, the most valuable features would be the Longitudinal Delta reporting, the PACS connectivity, and a larger organ coverage library. These features would enable CARVIS to evolve from a stand-alone case analysis application into an integrated "module" of the clinical image viewing workflow that could also be used for multi-visit oncology monitoring using the existing hospital IT systems. The Quest 3 client would benefit most from bimanual pinch-to-scale, runtime overlay of the source volume's segmentation mask, and a wrist-anchored hand-menu for transfer-function and case selection, essentially making the headset a self-contained pre-operative inspection station that would provide the AI results on-demand, across all three dimensions, in stereo 3D.

In the medium term, a prospective reader study will be required to quantify the benefit of the system on radiologist efficiency and reproducibility of measurements. This information will be important both for the scientific literature and to support the regulatory pathway necessary to release CARVIS as a certified medical device under the EU MDR or FDA SaMD standards. The addition of the CARVIS Q&A engine via the Quest's in headset microphone will enable surgeons to query the

grasped volume by voice and view the response as a HUD overlay. A blinded reader study comparing the headset client to flat panel review will also be necessary to facilitate broader adoption than early adopters.

Long term, this modular structure enables CARVIS to provide the infrastructure of a larger clinical AI ecosystem. Incremental inclusion of new imaging modalities (CT angiography, PET/CT), anatomies, and analytical functionality (radiomics feature extraction, treatment response prediction) becomes straightforward by adapting the pre-existing modules, rather than forcing a reworking of the platform. As LLM technology advances, and as local hardware optimization improves, Q&A and report generation modules stand to benefit and further narrow the gulf between automated analysis and the clinical report written by skilled radiologists today. The Color Passthrough feature of Quest 3 provides a tantalizing gateway toward true mixed-reality utilization within the operating room once the segmented volume is registered to the real patient, it can be visualized on location; and when combined with Meta XR co-presence, the same platform can facilitate remote tumor boards and surgical rehearsals in a shared three-dimensional environment.

Closing Remarks

The work at CARVIS illustrates that it is possible for a relatively small, tightly focused team to develop a clinically relevant, technically sound AI system by designing systems transparently, with modularity and end-user needs in mind. Documented contributions here validated segmentation pipelines, PCA measurement, atlas based localization, dice derived uncertainty and a guardrail enforced app stack all bring the discipline closer to trustworthy AI tools that refine rather than distort clinical judgment.

Health care represents one of the areas in which AI has the greatest potential to affect human well-being: shortening the interval between symptom and diagnosis, providing standardization of measures which is currently undermined by variability, and bringing specialist-level analysis to where specialists are in short supply. CARVIS is one such step in this process. The work presented here is merely a beginning, not the end, and the next steps outlined above show a clear route toward an implementable system in routine clinical practice.

4. References

1. Isensee, F., Jaeger, P. F., Kohl, S. A., Petersen, J., & Maier-Hein, K. H. (2021). nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. Nature Methods, 18(2), 203-211.

2. Menze, B. H., Jakab, A., Bauer, S., et al. (2015). The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS). IEEE Transactions on Medical Imaging, 34(10), 1993-2024.

3. Bakas, S., Akbari, H., Sotiras, A., et al. (2017). Advancing The Cancer Genome Atlas glioma MRI collections with expert segmentation labels and radiomic features. Scientific Data, 4, 170117.

4. Unity Technologies. (2023). Unity Real-Time Development Platform. Retrieved from https://unity.com

5. Meta Platforms, Inc. (2024). Meta Quest 3 Technical Specifications. Retrieved from https://www.meta.com/quest/quest-3/

6. Anwar, M., et al. (2023). Brain Tumor Segmentation in Multimodal MRI Using U-Net Layered Structure. Computers, Materials & Continua, 74(3), 5267–5281.

7. Wang, Z., Zhang, Z., Liu, J., & Yi, X. (2023). Research on Segmentation Method of Brain Tumor Image Based on Deep Learning. 2023 3rd International Conference on Electronic Information Engineering and Computer Science (EIECS), 118–122.

8. Yan, B.B. et al. (2022). MRI Brain Tumor Segmentation Using Deep Encoder-Decoder Convolutional Neural Networks. In: Crimi, A., Bakas, S. (eds) Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries. BrainLes 2021. Lecture Notes in Computer Science, vol 12963. Springer, Cham.

9. Zhang, F., Sun, Z., Wang, T. (2022). Brain Modeling for Surgical Training on the Basis of Unity 3D. In: Yang, S., Lu, H. (eds) Artificial Intelligence and Robotics. ISAIR 2022. Communications in Computer and Information Science, vol 1701. Springer, Singapore.

10. H. Gore, U. Chaskar, B. Borotikar and K. Bhole, "Medical Image Visualization using Augmented Reality," 2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI), Wardha, India, 2024, pp. 1-6

11. Abidin, A. Z., Naqvi, R. A., Haider, A., Kim, D. Y., Jeong, J. W., & Lee, S. W. (2024). Recent deep learning-based brain tumor segmentation models using multi-modality magnetic resonance imaging: a prospective survey. Frontiers in Bioengineering and Biotechnology, 12, 1392807.

12. Queisner, M., & Eisenträger, K. (2024). Surgical planning in virtual reality: a systematic review. Journal of Medical Imaging, 11(6), 062603.

13. Ahsan, R., Shahzadi, I., Najeeb, F., & Omer, H. (2024). Brain tumor detection and segmentation using deep learning. Magnetic Resonance Materials in Physics, Biology and Medicine, 38(1), 13-22.

14. Hoebel, K. V., Patel, J. B., Beers, A. L., et al. (2024). A review of deep learning for brain tumor analysis in MRI. npj Precision Oncology, 8, 5.

15. Peng, M. J., Chen, H. Y., Leung, F., et al. (2024). Virtual reality-based surgical planning simulator for tumorous resection in FreeForm Modeling: an illustrative case of clinical teaching. Quantitative Imaging in Medicine and Surgery, 14(2), 2060-2068.

16. Ujiie, H., Kato, T., Hu, H. P., et al. (2024). Developing a Virtual Reality Simulation System for Preoperative Planning of Robotic-Assisted Thoracic Surgery. Applied Sciences, 14(3), 973.

17. Bakhuis, W., Sadeghi, A. H., Moes, I., et al. (2023). Preparing for the future of cardiothoracic surgery with virtual reality simulation and surgical planning: a narrative review. Shanghai Chest, 7, 27.

18. Kersten-Oertel, M., Chen, S. J., Drouin, S., Sinclair, D. S., & Collins, D. L. (2019). Virtual interaction and visualisation of 3D medical imaging data with VTK and Unity. Healthcare Technology Letters, 6(1), 1-6.

19. Zhang, Y., Tang, W., Chen, S., et al. (2023). Online view enhancement for exploration inside medical volumetric data using virtual reality. Computers in Biology and Medicine, 163, 107193.

20. Khan, M. F., Iftikhar, A., Anwar, H., & Ramay, S. A. (2024). Brain Tumor Segmentation and Classification using Optimized Deep Learning. Journal of Computing & Biomedical Informatics, 7(01), 632-640.